

Data Structures

MCQ Corpus

Arrays · Linked Lists · Trees · Practice Sets

339 multiple-choice questions

Combined from 9 source sets

Table of Contents

Table of Contents	2
Arrays	3
From Forms	4
Grile Set 1	20
Grile Set 2	33
Grile Set 3	44
Grile Set 4	55
Linked Lists	68
SDD Test	69
Trees	84

Arrays

3 questions

Question 1 of 339

What is the time complexity of accessing an array element by index?

- A. **$O(1)$** ✓correct
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n \log n)$

Question 2 of 339

Which operation is expensive in a fixed-size array when inserting near the beginning?

- A. **Shifting following elements to make room** ✓correct
- B. Reading the first element
- C. Reading the array length
- D. Comparing two indexes

Question 3 of 339

What does contiguous storage usually improve?

- A. **Cache locality** ✓correct
- B. Pointer chasing
- C. Hash collisions
- D. Tree balancing

From Forms

74 questions

Question 4 of 339

For efficient deletion of arbitrary nodes, the most suitable structure is:

- A. Binary heap
- B. Doubly linked list ✓correct**
- C. Singly linked list
- D. Circular singly linked list
- E. Static array

Question 5 of 339

Select all correct Heap operations with $O(\log n)$ complexity:

- A. Inserting an element ✓correct**
- B. Traversing the entire heap
- C. Finding a random element
- D. Accessing the root element directly
- E. Deleting the root element ✓correct**

Question 6 of 339

If a node in a heap is at position "i", its left child is typically found at position:

- A. $2i - 1$
- B. $i/2$
- C. $i - 1$
- D. $2i + 2$
- E. $2i + 1$ ✓correct**

Question 7 of 339

Advantages of linked lists over heaps include:

- A. Faster random access
- B. Highest efficiency in sorting
- C. Able to preserve the order of the elements ✓correct**
- D. Direct access to min / max element
- E. Easier insertion/deletion operations at known positions ✓correct**

Question 8 of 339

Which data structure allows traversing in both forward and backward directions easily?

- A. Stacks
- B. Doubly linked list** ✓correct
- C. Queues
- D. Circular linked list
- E. Simple linked list

Question 9 of 339

Which is a correct description of a heap implementation:

- A. node processing based on the last updated element
- B. node processing based on the new element
- C. node processing based on the first inserted element
- D. node processing based on the priority of the element** ✓correct
- E. node processing based on the oldest element

Question 10 of 339

Operations on linked lists with $O(1)$ complexity (assuming direct node reference):

- A. Node insertion at known position** ✓correct
- B. Node deletion at known position** ✓correct
- C. Sorting the list
- D. Node search
- E. Traversing the entire list

Question 11 of 339

Deleting a node from a singly linked list, given the previous node, typically has complexity:

- A. $O(n^2)$
- B. $O(n \log n)$
- C. $O(n)$
- D. $O(1)$** ✓correct
- E. $O(\log n)$

Question 12 of 339

Which linked list variant doesn't have any node pointing to NULL?

- A. Doubly non-circular linked list
- B. Circular linked list** ✓correct
- C. Singly non-circular linked list
- D. Stack implemented as linked list
- E. Simple queue

Question 13 of 339

Select the true statements for linked lists and static arrays.

- A. List nodes are allocated at random addresses in the HEAP memory** ✓correct
- B. Static arrays have the memory allocated at runtime
- C. Accessing element i in a list is a $O(1)$ complexity time operation
- D. Memory size for static arrays is limited only by the HEAP size
- E. Lookup operation in an array is a $O(1)$ complexity time operation** ✓correct

Question 14 of 339

Removing the maximum element from a max-heap typically has complexity:

- A. $O(n \log n)$
- B. $O(1)$
- C. $O(n)$
- D. $O(\log n)$** ✓correct
- E. $O(n^2)$

Question 15 of 339

True statements about heaps:

- A. Allow fast random element access
- B. Insertion has $O(\log n)$ complexity** ✓correct
- C. Always sorted completely
- D. Efficiently implemented with arrays** ✓correct
- E. Can be either max-heap or min-heap** ✓correct

Question 16 of 339

A binary heap is considered a:

- A. Completely unordered structure
- B. Unstructured set of elements
- C. Circular structure
- D. Fully sorted structure
- E. Partially ordered structure** ✓correct

Question 17 of 339

Circular doubly linked list provides easy traversal in:

- A. Forward direction only
- B. Neither direction effectively
- C. Backward direction only
- D. Both forward and backward directions indefinitely** ✓correct
- E. Only in random directions

Question 18 of 339

Properties of binary heaps include:

- A. Always implemented via linked list
- B. Random access efficiency
- C. Complete binary tree structure ✓correct**
- D. Heap property for all nodes ✓correct**
- E. Nodes fully sorted

Question 19 of 339

Suitable for heap usage:

- A. Heap sort ✓correct**
- B. Priority queue ✓correct**
- C. Maintaining dynamic min/max ✓correct**
- D. Scheduling applications based on priority ✓correct**
- E. Direct indexing of arbitrary elements

Question 20 of 339

Benefits of doubly linked lists compared to singly linked lists include:

- A. Fixed memory allocation
- B. Easier node deletion when the node pointer is provided ✓correct**
- C. Less memory usage
- D. Easier random access
- E. Efficient backward traversal ✓correct**

Question 21 of 339

Main disadvantage of doubly linked lists:

- A. Unpredictable node addresses
- B. Increased memory overhead ✓correct**
- C. Difficult random access
- D. Difficult traversal
- E. Slow insertion

Question 22 of 339

Primary use of a min-heap:

- A. Linked list operations
- B. Fast random search
- C. Rapid traversal
- D. Accessing smallest element quickly ✓correct**

- E. Accessing largest element quickly

Question 23 of 339

After inserting a new element in a heap, the structure:

- A. Maintains heap property** ✓correct
- B. Becomes completely unstructured
- C. Is unchanged
- D. Randomly rearranges elements
- E. Becomes fully sorted

Question 24 of 339

What is the result of executing the following code?

- A. c = 1, d = 2, *pc = -10** ✓correct
- B. c = 1, d = 1, *pc = -10
- C. c = -10, d = 0, *pc = 1
- D. Application causes runtime error
- E. Application causes malloc() failure

Question 25 of 339

If an array is declared as `char mesaj[10]`, what is the valid index for the last element of the array?

- A. 9** ✓correct
- B. last
- C. `mesaj + 1`
- D. `mesaj - 1`
- E. 10

Question 26 of 339

Which function is used to dynamically allocate memory for a vector in C?

- A. malloc()** ✓correct
- B. `printf()`
- C. `sizeof()`
- D. `strlen()`
- E. `scanf()`

Question 27 of 339

Identify the problem in the code: `int* p; *p = 10;`

- A. Variable p is in read-only segment
- B. Pointer is constant

C. Memory not allocated before use ✓correct

D. Variable p contains null address

E. Cannot assign value 10

Question 28 of 339

Which data structure is used to implement the chaining mechanism in a hash table?

A. Tree structure

B. Heap structure

C. Matrix

D. Singly linked list ✓correct

E. Vector

Question 29 of 339

What types of rotations are used for AVL tree rebalancing?

A. Left-right rotation ✓correct

B. Single right rotation ✓correct

C. Right-left rotation ✓correct

D. Single left rotation ✓correct

Question 30 of 339

In which domains are trees used as applied data structures?

A. Defining a website's tree structure ✓correct

B. Organizing files in an operating system ✓correct

C. Organizing hierarchical structures in companies ✓correct

D. Implementing sequential calculations

Question 31 of 339

Which of the following are valid characteristics of statically allocated arrays?

A. Must be freed using free()

B. Vector size can be determined at runtime

C. Size is fixed ✓correct

D. Are allocated in the memory stack where they are declared ✓correct

E. Can be resized with realloc()

Question 32 of 339

What is true about the pointer variable p after calling free(p)?

A. It retains the content, i.e., the Heap memory address it had at allocation ✓correct

B. It is deallocated from the Stack segment where it was defined

C. It is copied to the Stack where it is accessed

- D. The pointer is automatically reset to NULL
- E. It is reset to a different Heap address

Question 33 of 339

What is the output of the following code?

- A. v[0] = 10, pv[0] = 10 ✓correct**
- B. Compilation error: index operator not applicable to pointers
- C. Compilation error: vector address cannot be assigned to pointer
- D. v[0] = 10, pv[0] = 0

Question 34 of 339

Which statements are true regarding singly linked lists?

- A. A singly linked list is a linear structure ✓correct**
- B. Nodes can be added/removed without knowing their count ✓correct**
- C. Nodes store heterogeneous data
- D. It allows sequential access ✓correct**
- E. Always has only two nodes that are homogeneous
- F. Each node stores multiple values of the same type

Question 35 of 339

What is the smallest value greater than 55 that can replace it in a BST with level-order: 55, 33, 77, 11, 44, 66, 88?

- A. 44
- B. 66 ✓correct**
- C. 77
- D. 88

Question 36 of 339

Which of the following are AVL tree rotations used to rebalance?

- A. Left-right rotation ✓correct**
- B. Right rotation ✓correct**
- C. Left rotation ✓correct**
- D. Right-left rotation ✓correct**

Question 37 of 339

Which of the following are true about pointer variables in C?

- A. Can be used in any arithmetic calculation
- B. Has special declaration syntax ✓correct**
- C. Stores a memory address ✓correct**

- D. Can only be used to access the data segment
- E. Can only be used to access heap memory

Question 38 of 339

Choose the correct code block to add a node with value 4 to the end of a singly linked list:

- A. Opțiunea 1
- B. Opțiunea 2
- C. **Opțiunea 3** ✓correct
- D. Opțiunea 4
- E. **Opțiunea 5** ✓correct

Question 39 of 339

What does chaining mean in a hash table?

- A. Using an auxiliary array
- B. Computing a hash function
- C. **Handling collisions through a list of elements** ✓correct
- D. Duplicating data sets in memory
- E. Translating the hash value into ASCII codes

Question 40 of 339

Which operation uses the Binary Search Tree data structure?

- A. **Clustering nodes based on a certain key value** ✓correct
- B. Transitive closure
- C. Topological sort
- D. Depth-first traversal

Question 41 of 339

What value is printed by this C code? `int v[] = {1, 2, 3, 4, 5}; printf("%d", v[3]);`

- A. 3
- B. **4** ✓correct
- C. 5
- D. 9

Question 42 of 339

Which of the following statements is true?

- A. **The function correctly implements the search operation for the parameter studName if it is a key in the hash table.** ✓correct
- B. The function incorrectly implements the search operation for the parameter studName if it is not a key in the hash table.

- C. The parameters of the insertStudent(struct Student*, int, struct Student) function are incorrectly declared in relation to how they are used.
- D. The function correctly implements the search operation for the parameter studName if it is not a key in the hash table.
- E. The function identifies all elements of type Student stored in the hash table.

Question 43 of 339

What is correct about searching in a hash table with chaining?

- A. Incorrect because hash allows direct access to a student list that caused a conflict
- B. Incorrect because hash generates values outside the allocated range
- C. Incorrect because hash cannot be used with string keys
- D. Correct because it uses only sequential access
- E. **Correct because it uses the hash function followed by sequential search** ✓correct

Question 44 of 339

Complete the code to correctly free memory and nullify the pointer:

- A. **free(v)** ✓correct
- B. **v = NULL** ✓correct

Question 45 of 339

What function is used to dynamically allocate memory for an array of elements in C?

- A. **malloc()** ✓correct
- B. printf()
- C. sizeof()
- D. strlen()
- E. scanf()

Question 46 of 339

Which of the following are situations where a right-left rotation is used in an AVL tree?

- A. Left imbalance with a right child
- B. Root has balance factor -2 and left child +1
- C. Balance factor is 0
- D. **Root has balance factor +2 and right child -1** ✓correct

Question 47 of 339

Select the characteristics of statically allocated arrays:

- A. Must be freed using free()
- B. Size can be determined at runtime
- C. **Size is fixed** ✓correct

D. Allocated on the stack ✓correct

E. Can be resized with realloc()

Question 48 of 339

What is the correct output of the following C code? `int v[] = {10, 20}; int* pv = v; printf("v[0] = %d, pv[0] = %d", v[0], pv[0]);`

A. v[0] = 10, pv[0] = 10 ✓correct

B. v[0] = 1, pv[0] = 20

C. v[0] = 10, pv[0] = -20

D. Compilation error due to [] operator

E. Compilation error due to pointer assignment

Question 49 of 339

What is the correct statement about the function:

A. The function inserts an element at the beginning of a singly linked list.

B. The function inserts a node at the beginning of a circular doubly linked list.

C. The head of the list is passed by value so that it is not updated.

D. The function returns the useful information stored in the head of the list. ✓correct

E. Deleting the container is equivalent to deleting the first ✓correct

Question 50 of 339

What are the advantages of linked lists over heaps?

A. Most efficient in sorting

B. Simplified dynamic memory management ✓correct

C. Faster random access

D. Easier insertions/deletions in known positions ✓correct

E. Direct access to min/max element

Question 51 of 339

What is true about pointer variable p after deallocating memory using free(p)?

A. It keeps the heap address it had at allocation ✓correct

B. It is deallocated from the stack

C. It is copied to the stack

D. It is automatically set to NULL

E. It is reset to another heap address

Question 52 of 339

What is true about the following C function using linear probing in a hash table?

- A. The function correctly implements the search operation for the studName parameter if it is a key in the hash table. ✓correct**
- B. The function incorrectly implements the search operation for the studName parameter if it is not a key in the hash table.
- C. The parameters of the function insertStudent(struct Student*, int, struct Student) are incorrectly declared relative to their usage.
- D. The function correctly implements the search operation for the studName parameter if it is not a key in the hash table.
- E. The function identifies all Student elements stored in the hash table.

Question 53 of 339

Identify the problem in the following code: `int* p; *p = 10;`

- A. Variable p is in read-only segment
- B. Pointer is constant
- C. Memory not allocated before use ✓correct**
- D. Variable p contains null address
- E. Cannot assign value 10

Question 54 of 339

What is the correct statement regarding the function:

- A. The function inserts an element at the beginning of a singly linked list.
- B. The function inserts a node at the beginning of a circular doubly linked list.
- C. The head of the list is passed by value in order not to update it.
- D. The function returns the useful information already present in the head of the list. ✓correct**
- E. The function deletes the container (node) corresponding to the first node in the list. ✓correct**

Question 55 of 339

What does the chaining mechanism in a hash table imply?

- A. Using an auxiliary array
- B. Computing a hash function
- C. Handling collisions through a list of elements ✓correct**
- D. Duplicating data in memory
- E. Translating hash into ASCII

Question 56 of 339

Which of the following statements are true about pointer variables in C?

- A. They can be used in any arithmetic calculation
- B. They have a special syntax for declaration ✓correct**
- C. They store a memory address ✓correct**

- D. Only used to access the data segment
- E. Only used to access heap memory

Question 57 of 339

What types of AVL rotations are used for balancing?

- A. **Left-right rotation** ✓correct
- B. **Right rotation** ✓correct
- C. **Left rotation** ✓correct
- D. **Right-left rotation** ✓correct

Question 58 of 339

Which operation uses the data structure “binary search tree”?

- A. **Clustering nodes based on a certain key value** ✓correct
- B. Transitive closure
- C. Topological sort
- D. Depth-first traversal

Question 59 of 339

Which data structure is used to implement chaining in a hash table?

- A. Tree structure
- B. Heap structure
- C. Matrix
- D. **Singly linked list** ✓correct
- E. Vector

Question 60 of 339

Which of the following are valid ways to declare and/or initialize a statically allocated two-dimensional (2D) array m in the C language?

- A. `int mat [2][3]={ (1, 2, 3},{4, 5, 6};`
- B. `int mat[][3] = {1, 2, 3},{4, 5, 6};`
- C. **`int mat[2][3] = {1, 2, 3, 4, 5, 6};`** ✓correct
- D. `int mat[2][]= {(1, 2, 3),(4, 5, 6)};`
- E. `int mat[][] = {(1, 2, 3),(4, 5, 6)};`

Question 61 of 339

Accessing the max element from a max heap typically has complexity :

- A. **O(1)** ✓correct
- B. O(N*N)
- C. O(N)

- D. $O(N \log N)$
- E. $O(\log N)$

Question 62 of 339

What type of tree traversal is used in the saveflightsVector function?

- A. postorder
- B. inorder
- C. **preorder** ✓correct
- D. breadth-first

Question 63 of 339

If a node in a heap is at position "i" its left child is typically found at position:

- A. $-2i + 2$
- B. $i/2$
- C. $2i - 1$.
- D. **$2i + 1$** ✓correct
- E. $i - 1$

Question 64 of 339

Which memory segment is used for allocating local variables?

- A. BSS (Block Started by Symbol)
- B. **Stack** ✓correct
- C. Data
- D. Heap
- E. Code

Question 65 of 339

A singly linked list is best suited for:

- A. Random access
- B. Fixed-size data storage
- C. c Frequent random searching
- D. Binary searches
- E. **Sequential access and frequent modification** ✓correct

Question 66 of 339

Which data structure allows traversing in both forward and backward directions easily?

- A. Stacks
- B. **Doubly linked list** ✓correct
- C. Circular linked list

- D. Queues
- E. Simple linked list

Question 67 of 339

What role does the parameter `int* nr` in the `saveflightsvector` function?

- A. Remembers the address of the tree for traversal
- B. Checks if the flight year is valid
- C. **Remembers the number of flights saved in the vector** ✓correct
- D. Stores the code of the last flight inserted into the vector

Question 68 of 339

A heap is usually visualized as:

- A. Array of pointers
- B. **Binary Tree** ✓correct
- C. Graph
- D. Linked list
- E. Hash table

Question 69 of 339

After inserting a new element in a heap, the structure:

- A. is unchanged
- B. Becomes fully sorted
- C. Becomes completely unstructured
- D. **Maintains heap property** ✓correct
- E. Randomly rearranges elements

Question 70 of 339

In a circular singly linked list, the last node points to:

- A. Second node
- B. **First node** ✓correct
- C. Itself
- D. A random middle node
- E. NULL

Question 71 of 339

In a binary search tree (BST) that maintains the total order relation (left key < parent key < right key), if a node that has two subtrees needs to be deleted, what steps are taken?

- A. **The maximum value in the left subtree is located.** ✓correct
- B. The maximum value in the right subtree is located.

- C. The node is deleted directly without any additional actions.
- D. The rightmost node in the left subtree is located. ✓correct**
- E. The minimum value in the right subtree is located. ✓correct**

Question 72 of 339

Identify the true statements about singly linked lists.

- A. Allows quick random access
- B. Efficient backward traversal
- C. Each node has exactly two pointers
- D. Efficient insertion if node position is known ✓correct**
- E. Memory allocation is dynamic ✓correct**

Question 73 of 339

In the postorder traversal of a tree, the order of visiting is:

- A. Current node - subtrees
- B. Current node - leaves
- C. Right subtree - left subtree → current node
- D. Left subtree - right subtree - current node ✓correct**
- E. Current node - right subtree - left subtree

Question 74 of 339

Where the int data type is a long(4 bytes per int)?

- A. 0x00C7FED8 ✓correct**

Question 75 of 339

Arrange the C code:

- A. i=0; ✓correct**
- B. ht[i]=s ✓correct**

Question 76 of 339

What is wrong with the following code snippet?

- A. No pointer was used for dynamic allocation
- B. Exceeds the arrays bounds ✓correct**
- C. The element at position i is accessed incorrectly using the [] operator
- D. Allocation error because realloc() should be used instead of malloc()
- E. The variable i was not declared

Question 77 of 339

Which are correct?

- A. deletion of an element from a list
- B. adding a node at the end of the list** ✓correct
- C. adding a node at the begging of the list
- D. searching for a node with payload info
- E. inserting into a circular list** ✓correct

Grile Set 1

47 questions

Question 78 of 339

Ce presupune mecanismul de tip chaining intr-o tabela de dispersie?

- A. Folosirea unui vector auxiliar.
- B. Calcularea unei functii de dispersie.
- C. Tratarea coliziunilor printr-o lista de elemente. ✓correct**
- D. Duplicarea seturilor de date in memorie.
- E. Translatarea in coduri ASCII a valorii hash.

Question 79 of 339

Ce structura de date se utilizeaza pentru implementarea mecanismului chaining pentru o tabela de dispersie?

- A. Structura arborescenta.
- B. Structura Heap.
- C. Matrice.
- D. Lista simplu inlantuita. ✓correct**
- E. Vector.

Question 80 of 339

Care dintre urmatoarele afirmatii este corecta? (se considera secventa de cod C pentru cautare student intr-o tabela de dispersie implementata cu mecanism chaining, avand numele studentului cu rol de cheie) Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> struct Student { int id; char* nume; float medie; }; struct Nod { struct Student st; struct Nod* next; }; int calculPozitie(char* key, int size) { int temp = 0; for (unsigned int i = 0; i < strlen(key); i++) temp += key[i]; return (temp % size); } struct Student cautaHTable(struct Nod** hTab, int size, char* nume_student) { int k = calculPozitie(nume_student, size); struct Student stud; stud.nume = NULL; struct Nod* tmp = hTab[k]; while (tmp) { if (!strcmp(tmp->st.nume, nume_student)) stud = tmp->st; tmp = tmp->next; } return stud; }

- A. Implementarea operatiei de cautare este incorecta deoarece functia hash permite accesul direct la lista de studenti care au generat conflict pe aceeasi pozitie.
- B. Implementarea operatiei de cautare este incorecta deoarece functia hash genereaza valori plasate in afara zonei alocate pentru tabela de dispersie.
- C. Implementarea operatiei de cautare este incorecta deoarece functia hash nu poate fi utilizata pentru nume student.
- D. Implementarea operatiei de cautare este corecta deoarece se utilizeaza exclusiv accesul secvential la datele stocate in tabela de dispersie.**

E. Implementarea operatiei de cautare este corecta deoarece se utilizeaza functia hash urmata de cautarea secventiala a studentului. ✓correct

Question 81 of 339

Precizati rezultatul executiei aplicatiei de mai sus: Code: #include <stdio.h> #include <malloc.h> int main() { int* pc, c, d; pc = (int*)malloc(sizeof(int)); pc[0] = -10; c = *pc + 11; d = c + 1; printf("c = %d, d = %d, *pc = %d\n", c, d, *pc); free(pc); return 0; }

- A. c = 1, d = 2, *pc = -10** ✓correct
- B. c = 1, d = 1, *pc = -10
- C. Aplicatia determina eroare de compilare deoarece variabilele c si d nu sunt modificate corect cu valorile calculate pe baza adresei de memorie heap stocata in variabila pointer pc.
- D. c = -10, d = -9, *pc = -10
- E. Aplicatia determina eroare de executie deoarece nu se aloca suficient spatiu de memorie heap la apelul functiei malloc().

Question 82 of 339

Pentru stergere eficienta a nodurilor arbitrare, cea mai potrivita structura este:

- A. Heap binar
- B. Lista dublu inlantuita** ✓correct
- C. Lista simplu inlantuita
- D. Lista circulara simplu inlantuita
- E. Tablou static

Question 83 of 339

Care dintre urmatoarele afirmatii sunt adevarate, cu privire la urmatoarea secventa de cod sursa C: (creeaza vector de 100 de elemente pentru tabela de dispersie cu chaining) Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> #define DIM 100 struct Student { int id; char* nume; float medie; }; struct Nod { struct Student student; struct Nod* next; }; struct Nod** hTable; struct Student* stud; hTable = (struct Nod**)malloc(sizeof(struct Nod*) * DIM); for (int i = 0; i < DIM; i++) hTable[i] = NULL;

- A. Initializarea elementelor din vector este efectuata cu valoarea unui pointer nul.** ✓correct
- B. Initializeaza un vector de 100 de elemente de tip Nod*.** ✓correct
- C. Aloca si initializeaza structura suport pentru o tabela de dispersie care implementeaza mecanismul de tip chaining pentru rezolvarea cazurilor de coliziune.** ✓correct
- D. Aloca un vector cu dimensiunea 100 de elemente.** ✓correct
- E. Initializeaza un vector de 100 de elemente de tip Nod**.

Question 84 of 339

Completati codul pentru eliberarea corecta spatiului ocupat de un vector alocat dinamic: Code: int nrEl = 10; int* v = malloc(nrEl * sizeof(int)); for (int i = 0; i < nrEl; i++) { v[i] = i; } _____(v); v = _____;

- A. delete
- B. clear
- C. free ✓correct
- D. clean
- E. NULL ✓correct

Question 85 of 339

Care dintre urmatoarele afirmatii este adevarata? (se considera secventa de cod C pentru o tabela de dispersie implementata cu linear probing) Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> #define ARRAY_SIZE 100 struct Student { int id; char* nume; float avg; }; int insertStudent(struct Student* h, int size, char* studName) { char found = 0; i; for (i = 0; i < size && !found; i++) { if ((h[i].nume != 0) && (strcmp(h[i].nume, studName) == 0)) { found = 1; } } if (found == 1) return 1; else return -1; } struct Student* HTable; int size = ARRAY_SIZE; HTable = (struct Student*)malloc(size * sizeof(struct Student)); for (int i = 0; i < size; i++) { HTable[i].nume = NULL; }

- A. Functia implementeaza corect operatia de cautare pentru parametrul studName daca acesta este cheie pentru tabela de dispersie. ✓correct
- B. Functia implementeaza incorect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- C. Parametrii functiei insertStudent(struct Student*, int, struct Student) sunt declarati eronat in raport cu modul lor de utilizare.
- D. Functia implementeaza corect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- E. Functia identifica toate elementele de tip Student stocate in tabela de dispersie.

Question 86 of 339

Daca un vector este declarat ca char mesaj[10] care este indexul valid pentru ultimul element al acestui vector?

- A. 9 ✓correct
- B. last
- C. mesaj +1
- D. mesaj -1
- E. 10

Question 87 of 339

Care dintre urmatoarele sunt cazuri in care se foloseste rotatia dreapta-stanga intr-un arbore AVL?

- A. Dezechilibru la stanga cu fiu dreapta
- B. Radacina are gradul de echilibru -2 si fiul stanga +1
- C. Gradul de echilibru este 0

D. Radacina are gradul de echilibru +2 si fiul dreapta -1 ✓correct

Question 88 of 339

Ce funcție este folosită pentru a alocă memorie în mod dinamic pentru un vector de elemente în limbajul C?

A. malloc() ✓correct

B. printf()

C. sizeof()

D. strlen()

E. scanf()

Question 89 of 339

Care dintre următoarele afirmații sunt adevărate cu privire la o structură de tip listă simplă?

A. Lista simplă este o structură liniară. ✓correct

B. Lista simplă este o structură dinamică în sensul că nodurile pot fi adăugate și/sau eliminate oricând fără a se cunoaște apriori numărul acestora. ✓correct

C. Lista simplă conține noduri unde două noduri diferite sunt eterogene din punct de vedere al datelor stocate.

D. Lista simplă este o structură a cărei elemente sunt referite în acces secvențial. ✓correct

E. Lista simplă conține noduri unde două noduri diferite sunt omogene din punct de vedere al datelor stocate. ✓correct

F. Lista simplă conține noduri, iar fiecare nod stochează mai multe elemente de același tip.

Question 90 of 339

Avantaje ale listelor înlănțuite față de heap-uri:

A. Cea mai mare eficiență în sortare

B. Management dinamic simplificat în anumite contexte ✓correct

C. Acces aleator mai rapid

D. Inserții/ștergeri mai simple în poziții cunoscute ✓correct

E. Acces direct la elementul min / max

Question 91 of 339

Identificați problema secvenței de cod sursă următoare: Code: `int* p; *p = 10;`

A. Variabila `p` este alocată în segmentul de cod (read-only), deci nu poate fi modificată.

B. Pointerul este constant.

C. Memoria nu este alocată înainte de utilizare. ✓correct

D. Variabila `p` conține adresa nulă.

E. Nu se poate modifica valoarea 10.

Question 92 of 339

Se considera urmatoarea secventa de cod sursa C pentru o tabela de dispersie implementata cu linear probing: Care dintre urmatoarele afirmatii este adevarata? Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> #define ARRAY_SIZE 100 struct Student { int id; char* nume; float avg; }; int functiaStudent(struct Student* h, int size, char* studName) { char found = 0, i; for (i = 0; i < size && !found; i++) { if ((h[i].nume != 0) && (strcmp(h[i].nume, studName) == 0)) { found = 1; } } if (found == 1) return i; else return -1; } struct Student* HTable; int size = ARRAY_SIZE; HTable = (struct Student*)malloc(size * sizeof(struct Student)); for (int i = 0; i < size; i++) { HTable[i].nume = NULL; }

- A. Functia implementeaza corect operatia de cautare pentru parametrul studName daca acesta este cheie pentru tabela de dispersie.** ✓correct
- B. Functia implementeaza incorect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- C. Parametrii functiei insertStudent(struct Student*, int, struct Student) sunt declarati eronat in raport cu modul lor de utilizare.
- D. Functia implementeaza corect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- E. Functia identifica toate elementele de tip Student stocate in tabela de dispersie.

Question 93 of 339

Selectați caracteristicile vectorilor alocați în mod static:

- A. Trebuie să fie eliberați cu ajutorul funcției free()
- B. Dimensiunea vectorului poate fi stabilită la momentul execuției
- C. Dimensiunea este fixă** ✓correct
- D. Sunt alocați pe stiva de memorie a funcției în care sunt declarați** ✓correct
- E. Pot fi redimensionați cu ajutorul funcției realloc()

Question 94 of 339

În ce domenii se folosesc arborii ca structuri de date aplicate? Selectați unul sau mai multe:

- A. Definirea structurii arborescente a unui site web** ✓correct
- B. Organizarea fișierelor în cadrul unui sistem de operare** ✓correct
- C. Organigrame sau structuri ierarhice în companii** ✓correct
- D. Implementarea căutărilor secvențiale

Question 95 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C: Precizati rezultatul executiei aplicatiei de mai sus: Code: #include <stdio.h> #include <malloc.h> int main() { int v[] = {10, 20}; int *pv = v; printf("v[0] = %d, pv[0] = %d\n", v[0], pv[0]); return 0; }

- A. v[0] = 10, pv[0] = 20
- B. v[0] = 10, pv[0] = 0

- C. Aplicatia determina eroare de compilare deoarece operatorul de indexare [] nu se aplica variabilei de tip pointer.
- D. Aplicatia determina eroare de compilare deoarece adresa unei variabile vector nu poate fi atribuita unei variabile de tip pointer.
- E. **v[0] = 10, pv[0] = 10 ✓ correct**

Question 96 of 339

Se considera urmatoarea secventa de cod sursa C cu privire la operatia de cautare a unui student intr-o tabela de dispersie implementata cu mecanismul de tip chaining pentru tratarea coliziunilor avand numele studentului cu rol de cheie: Care dintre urmatoarele afirmatii este corecta? Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> struct Student { int id; char* nume; float medie; }; struct Nod { struct Student st; struct Nod* next; }; int calculPozitie(char* key, int size) { int temp = 0; for (unsigned int i = 0; i < strlen(key); i++) temp += key[i]; return (temp % size); } struct Student cautInTable(struct Nod** hTab, int size, char* nume_student) { int k = calculPozitie(nume_student, size); struct Student stud; stud.nume = NULL; struct Nod* tmp = hTab[k]; while (tmp) { if (!strcmp(tmp->st.nume, nume_student)) stud = tmp->st; tmp = tmp->next; } return stud; }

- A. Implementarea operatiei de cautare este incorecta deoarece functia hash permite accesul direct la lista de studenti care au generat conflict pe cheia de cautare.
- B. Implementarea operatiei de cautare este incorecta deoarece functia hash genereaza valori plasate in afara zonei alocate pentru tabela de dispersie.
- C. Implementarea operatiei de cautare este incorecta deoarece functia hash nu poate fi utilizata pentru nume student.
- D. Implementarea operatiei de cautare este corecta deoarece se utilizeaza exclusiv accesul secvential la datele stocate in tabela de dispersie.
- E. **Implementarea operatiei de cautare este corecta deoarece se utilizeaza functia hash urmata de cautarea secventiala a studentului. ✓ correct**

Question 97 of 339

Care afirmatie este adevarata cu privire la variabila pointer p la momentul dealocarii memoriei Heap prin apelarea functiei free(p)?

- A. **Își păstrează conținutul, respectiv adresa de memorie Heap obținută la momentul alocării. ✓ correct**
- B. Se dezaloca variabila p de pe segmentul de Stack unde a fost alocata la momentul definirii sale.
- C. Se copiaza datele in memorie Stack unde vor fi accesate prin variabila pointer p.
- D. Se reinitializeaza cu pointerul nul.
- E. Se rescrie cu o adresa de memorie Heap diferita.

Question 98 of 339

Ce presupune mecanismul de tip chaining intr-o tabela de dispersie?

- A. Folosirea unui vector auxiliar.

- B. Calcularea unei functii de dispersie.
- C. Tratarea coliziunilor printr-o lista de elemente. ✓correct**
- D. Duplicarea seturilor de date in memorie.
- E. Translatarea in coduri ASCII a valorii hash.

Question 99 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o variabila de tip pointer in limbajul C?

- A. Poate fi utilizata in orice calcul aritmetic.
- B. Prezinta o sintaxa dedicata pentru declarare. ✓correct**
- C. Stocheaza o adresa de memorie. ✓correct**
- D. Poate fi utilizata doar pentru accesul la segmente de date.
- E. Poate fi utilizata doar pentru accesul la memorie heap.

Question 100 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la urmatoarea secventa de cod sursa C: Code: #include <stdio.h> #include <malloc.h> #define DIM 100 #define SPERS NULL struct Student { int id; char* name; float media; }; struct Nod { struct Student s; struct Nod* next; }; struct Nod** stTable; struct Student stKey; stTable = (struct Nod**)malloc(sizeof(struct Nod*) * DIM); for (int i = 0; i < DIM; i++) stTable[i] = 0;

- A. Initializarea elementelor din vector este efectuata cu valoarea unui pointer nul. ✓correct**
- B. Initializeaza un vector de 100 de elemente de tip Nod*. ✓correct**
- C. Aloca si initializeaza structura suport pentru o tabela de dispersie care implementeaza mecanismul de tip chaining pentru rezolvarea cazurilor de coliziune. ✓correct**
- D. Aloca un vector cu dimensiunea 100 de elemente. ✓correct**
- E. Initializeaza un vector de 100 de elemente de tip Nod**.

Question 101 of 339

Ce tipuri de rotații se aplică pentru reechilibrarea unui arbore AVL?

- A. Rotație stânga-dreapta ✓correct**
- B. Rotație stânga ✓correct**
- C. Rotație dreapta-stânga ✓correct**
- D. Rotație dreapta ✓correct**
- E. N/A

Question 102 of 339

Care din valorile de mai jos poate fi cheie pentru nodul radacina dupa stergerea nodului cu cheia 55 din arborele binar de cautare a carui reprezentare pe niveluri este 55-33-77-11-44-66-88?

- A. 44 ✓correct**

B. 66 ✓correct

C. 77

D. 88

E. N/A

Question 103 of 339

Care este afirmatia corecta cu privire la urmatoarea functie? Code: Student*

```
functionList(Node** head) {
    Student* student = NULL;
    if (*head != NULL) {
        student = (*head)->info;
        Node* tmp = *head;
        if ((*head)->next != (*head)) {
            tmp->prev->next = tmp->next;
            tmp->next->prev = tmp->prev;
            *head = tmp->next;
        }
        else
            *head = NULL;
        free(tmp);
    }
    return student;
}
```

A. functia insereaza un element la inceputul unei liste simplu inlantuite

B. functia insereaza un nod la inceputul unei liste dublu inlantuite

C. capatul listei este trimis prin valoare pentru a nu-l actualiza

D. functia returneaza informatia utila existenta in capatul listei ✓correct

E. stergerea containerului echivalent primul nod din lista ✓correct

Question 104 of 339

Ce valoare este afisata in urma executiei secventei de cod de mai jos? Code: int v[5] = { 1, 2, 3, 4, 5 }; printf("%d", v[3]);

A. 5

B. 2

C. 4 ✓correct

D. 3

E. 1

Question 105 of 339

Considerand ca urmatoarea secventa care de mai jos reprezinta procedeul utilizat de catre nodul de valoare 4 la sfarsitul secventei curente: Code: // Existing list: 3 -> 2 -> 1 struct node* current = head; while (current->next != NULL) { current = current->next; } // Add code here to append a new node with value 4

A. current->info = 4;

B. current->next->next = malloc(sizeof(Node)); current->next->next->info = 4;

C. current->next = malloc(sizeof(Node)); current->next->info = 4; current->next->next = NULL; ✓correct

D. current = malloc(sizeof(Node)); current->info = 4;

E. current->next = malloc(sizeof(Node)); current = current->next; current->info = 4; current->next = NULL; ✓correct

Question 106 of 339

Ce structura de date se utilizeaza pentru implementarea mecanismului chaining pentru o tabela de dispersie?

- A. Structura arborescenta.
- B. Structura Heap.
- C. Matrice.
- D. Lista simplu inlantuita. ✓correct**
- E. Vector.

Question 107 of 339

Operatii aplicabile arborilor binari de cautare includ:

- A. Cautare nod dupa o anumita valoare a cheii ✓correct**
- B. Traversare postordine ✓correct**
- C. Sortare topologica
- D. Inserare nod nou ✓correct**
- E. N/A

Question 108 of 339

Considerându-se următoarea secvență, care dintre fragmentele procedurale adaugă un nou nod la sfârșitul listei? Code: // Existing list: 9 -> 2 -> 3 struct Node* current = head; while (current->next != NULL) { current = current->next; } // Add code here to append a new node with value 4

- A. current->info = 4;
- B. current->next->next = malloc(sizeof(Node)); current->next->next->info = 4;
- C. current->next = malloc(sizeof(Node)); current->next->info = 4; current->next->next = NULL; ✓correct**
- D. current = malloc(sizeof(Node)); current->info = 4;
- E. current->next = malloc(sizeof(Node)); current = current->next; current->info = 4; current->next = NULL; ✓correct**

Question 109 of 339

Care este afirmația corectă cu privire la următoarea funcție? Code: Student* functionList(Node** head) { Student* student = NULL; if (*head != NULL) { student = (*head)->info; Node* tmp = *head; if ((*head)->next != (*head)) { tmp->prev->next = tmp->next; tmp->next->prev = tmp->prev; *head = tmp->next; } else *head = NULL; free(tmp); } return student; }

- A. funcția insereaza un element la inceputul unei liste simplu inlantuite
- B. funcția insereaza un nod la inceputul unei liste dublu inlantuite circulare
- C. capatul listei este trimis prin valoare pentru a nu-l actualiza
- D. funcția returneaza informatia utila existenta in capatul listei ✓correct**
- E. stergerea containerului echivalent primul nod din lista ✓correct**

Question 110 of 339

Care din valorile de mai jos poate fi cheia pentru nodul radacina după ștergerea nodului cu cheia 55 din arborele binar de cautare a carui reprezentare pe niveluri este 55-33-77-11-44-66-88?

- A. 44 ✓correct
- B. 66 ✓correct
- C. 77
- D. 88

Question 111 of 339

Care dintre următoarele afirmații sunt adevărate cu privire la o variabilă de tip pointer în limbajul C?

- A. Poate fi utilizată în orice calcul aritmetic.
- B. Prezintă o sintaxă dedicată pentru declarare. ✓correct
- C. Stochează o adresă de memorie. ✓correct
- D. Poate fi utilizată doar pentru accesul la segmente de date.
- E. Poate fi utilizată doar pentru accesul la memorie heap.

Question 112 of 339

În ce domenii se folosesc arborii ca structuri de date aplicate?

- A. Definirea structurii arborescente a unui site web ✓correct
- B. Organizarea fișierelor în cadrul unui sistem de operare ✓correct
- C. Organigrame sau structuri ierarhice în companii ✓correct
- D. Implementarea căutărilor secvențiale

Question 113 of 339

Ce tipuri de rotații se aplică pentru reechilibrarea unui arbore AVL?

- A. Rotație stânga-dreapta ✓correct
- B. Rotație stânga ✓correct
- C. Rotație dreapta-stânga ✓correct
- D. Rotație dreapta ✓correct

Question 114 of 339

Care dintre următoarele afirmații sunt adevărate cu privire la o structură de tip listă simplă?

- A. Lista simplă este o structură liniară. ✓correct
- B. Lista simplă este o structură dinamică în sensul că nodurile pot fi adăugate și/sau eliminate oricând fără a se cunoaște apriori numărul acestora. ✓correct
- C. Lista simplă conține noduri unde două noduri diferite sunt eterogene din punct de vedere al datelor stocate.

- D. Lista simpla este o structura a carei elemente sunt referite in acces secvential.** ✓correct
- E. Lista simpla contine noduri unde doua noduri diferite sunt omogene din punct de vedere al datelor stocate.** ✓correct
- F. Lista simpla contine noduri, iar fiecare nod stocheaza mai multe elemente de acelasi tip.

Question 115 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: `#include <stdio.h> #include <malloc.h> int main() { int v[] = {10, 20}; int *pv = v; printf("v[0] = %d, pv[0] = %d\n", v[0], pv[0]); return 0; }`

- A. v[0] = 10, pv[0] = 20
- B. v[0] = 10, pv[0] = 0
- C. Aplicatia determina eroare de compilare deoarece operatorul de indexare [] nu se aplica variabilei de tip pointer.
- D. Aplicatia determina eroare de compilare deoarece adresa unei variabile vector nu poate fi atribuita unei variabile de tip pointer.
- E. v[0] = 10, pv[0] = 10** ✓correct

Question 116 of 339

Identificati problema secventei de cod sursa urmatoare: Code: `int* p; *p = 10;`

- A. Variabila p este alocata in segmentul de cod (read-only), deci nu poate fi modificata.
- B. Pointerul este constant.
- C. Memoria nu este alocata inainte de utilizare.** ✓correct
- D. Variabila p contine adresa nula.
- E. Nu se poate modifica valoarea 10.

Question 117 of 339

Care afirmatie este adevarata cu privire la variabila pointer p la momentul dezalocarii memoriei Heap prin apelarea functiei free(p)?

- A. Isi pastreaza continutul, respectiv adresa de memorie Heap obtinuta la momentul alocarii.** ✓correct
- B. Se dezaloca variabila p de pe segmentul de Stack unde a fost alocata la momentul definirii sale.
- C. Se copiaza datele in memorie Stack unde vor fi accesate prin variabila pointer p.
- D. Se reinitializeaza cu pointerul nul.
- E. Se rescrie cu o adresa de memorie Heap diferita.

Question 118 of 339

Selectează caracteristicile vectorilor alocați în mod static:

- A. Trebuie să fie eliberați cu ajutorul funcției free()
- B. Dimensiunea vectorului poate fi stabilită la momentul execuției
- C. Dimensiunea este fixă** ✓correct

D. Sunt alocați pe stiva de memorie a funcției în care sunt declarați ✓correct

E. Pot fi redimensionați cu ajutorul funcției realloc()

Question 119 of 339

Avantaje ale listelor înlănțuite față de heap-uri:

A. Cea mai mare eficiență în sortare

B. Management dinamic simplificat în anumite contexte ✓correct

C. Acces aleator mai rapid

D. Inserții/ștergeri mai simple în poziții cunoscute ✓correct

E. Acces direct la elementul min / max

Question 120 of 339

Completează codul pentru eliberarea corectă spațiului ocupat de un vector alocat dinamic:

Code: `int nrEl = 10; int* v = malloc(nrEl * sizeof(int)); for (int i = 0; i < nrEl; i++) { v[i] = i; }
____(v); v = ____;`

A. delete

B. clear

C. free ✓correct

D. clean

E. NULL ✓correct

Question 121 of 339

Care dintre următoarele sunt cazuri în care se folosește rotația dreapta-stânga într-un arbore AVL?

A. Dezechilibru la stânga cu fiu drept

B. Rădăcina are gradul de echilibru -2 și fiul stânga +1

C. Gradul de echilibru este 0

D. Rădăcina are gradul de echilibru +2 și fiul dreapta -1 ✓correct

E. (none)

Question 122 of 339

Pentru ștergere eficientă a nodurilor arbitrare, cea mai potrivită structură este:

A. Heap binar

B. Listă dublu înlănțuită ✓correct

C. Listă simplu înlănțuită

D. Listă circulară simplu înlănțuită

E. Tablou static

Question 123 of 339

Dacă un vector este declarat ca `char mesaj[10]` care este indexul valid pentru ultimul element al acestui vector?

- A. 9 ✓ *correct*
- B. last
- C. mesaj + 1
- D. mesaj - 1
- E. 10

Question 124 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: `#include <stdio.h> #include <malloc.h> int main() { int* pc, c, d; pc = (int*)malloc(sizeof(int)); pc[0] = -10; c = *pc + 11; d = c + 1; printf("c = %d, d = %d, *pc = %d\n", c, d, *pc); free(pc); return 0; }`

- A. c = 1, d = 2, *pc = -10 ✓ *correct*
- B. c = 1, d = 1, *pc = -10
- C. Aplicatia determina eroare de compilare deoarece variabilele c si d nu sunt modificate corect cu valorile calculate pe baza adresei de memorie heap stocata in variabila pointer
- D. c = -10, d = -9, *pc = -10
- E. Aplicatia determina eroare de executie deoarece nu se alocă suficient spatiu de memorie heap la apelul functiei malloc().

Grile Set 2

44 questions

Question 125 of 339

Dacă un vector este declarat ca `char mesaj[10]` care este indexul valid pentru ultimul element al acestui vector?

- A. 9 ✓correct
- B. last
- C. mesaj +1
- D. mesaj - 1
- E. 10

Question 126 of 339

Care dintre următoarele sunt cazuri în care se folosește rotația dreapta-stânga într-un arbore AVL? Selectați unul sau mai multe:

- A. Dezechilibru la stânga cu fiu dreapta
- B. Rădăcina are gradul de echilibru -2 și fiul stânga +1
- C. Gradul de echilibru este 0
- D. Rădăcina are gradul de echilibru +2 și fiul dreapta -1 ✓correct

Question 127 of 339

Care dintre urmatoarele afirmatii este corecta? Code: `#include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> struct Student { int id; char* nume; float medie; }; struct Nod { struct Student st; struct Nod* next; }; int calculPozitie(char* key, int size) { int temp = 0; for (unsigned int i = 0; i < strlen(key); i++) temp += key[i]; return (temp % size); } struct Student cautaHTable(struct Nod** hTab, int size, char* nume_student) { int k = calculPozitie(nume_student, size); struct Student stud; stud.nume = NULL; struct Nod* tmp = hTab[k]; while (tmp) { if (!strcmp(tmp->st.nume, nume_student)) stud = tmp->st; tmp = tmp->next; } return stud; }`

- A. Implementarea operatiei de cautare este incorecta deoarece functia hash permite accesul direct la lista de studenti care au generat conflict pe aceeasi pozitie.
- B. Implementarea operatiei de cautare este incorecta deoarece functia hash genereaza valori plasate in afara zonei alocate pentru tabela de dispersie.
- C. Implementarea operatiei de cautare este incorecta deoarece functia hash nu poate fi utilizata pentru nume student.
- D. Implementarea operatiei de cautare este corecta deoarece se utilizeaza exclusiv accesul secvential la datele stocate in tabela de dispersie.
- E. Implementarea operatiei de cautare este corecta deoarece se utilizeaza functia hash urmata de cautarea secventiala a studentului. ✓correct

Question 128 of 339

Pentru ștergere eficientă a nodurilor arbitrare, cea mai potrivită structură este:

- A. Heap binar
- B. Listă dublu înlănțuită ✓ correct**
- C. Listă simplu înlănțuită
- D. Listă circulară simplu înlănțuită
- E. Tablou static

Question 129 of 339

Completează codul pentru eliberarea corectă a spațiului ocupat de un vector alocat dinamic

Code: `int nrEl = 10; int* v = malloc(nrEl * sizeof(int)); for (int i = 0; i < nrEl; i++) { v[i] = i; }
 ____ (v); v = ____;`

- A. delete
- B. clear
- C. free ✓ correct**
- D. clean
- E. NULL ✓ correct**
- F. malloc(nrEl)

Question 130 of 339

Selectează caracteristicile vectorilor alocați în mod static:

- A. Trebuie să fie eliberați cu ajutorul funcției free()
- B. Dimensiunea vectorului poate fi stabilită la momentul execuției
- C. Dimensiunea este fixă ✓ correct**
- D. Sunt alocați pe stiva de memorie a funcției în care sunt declarați ✓ correct**
- E. Pot fi redimensionați cu ajutorul funcției realloc()

Question 131 of 339

Avantaje ale listelor înlănțuite față de heap-uri:

- A. Cea mai mare eficiență în sortare
- B. Management dinamic simplificat în anumite contexte ✓ correct**
- C. Acces aleator mai rapid
- D. Inserții/ștergeri mai simple în poziții cunoscute ✓ correct**
- E. Acces direct la elementul min / max

Question 132 of 339

Precizați rezultatul executiei aplicatiei de mai sus: Code: `#include <stdio.h> #include`

`<malloc.h> int main() { int* pc, c, d; pc = (int*)malloc(sizeof(int)); pc[0] = -10; c = *pc + 11; d = c + 1; printf("c = %d, d = %d, *pc = %d\n", c, d, *pc); free(pc); return 0; }`

- A. **c = 1, d = 2, *pc = -10** ✓correct
- B. c = 1, d = 1, *pc = -10
- C. Aplicatia determina eroare de compilare deoarece variabilele c si d nu sunt modificate corect cu valorile calculate pe baza adresei de memorie heap stocata in variabila pointer.
- D. c = -10, d = -9, *pc = -10
- E. Aplicatia determina eroare de executie deoarece nu se aloca suficient spatiu de memorie heap la apelul functiei malloc().

Question 133 of 339

Care dintre urmatoarele afirmatii este adevarata? Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> #define ARRAY_SIZE 100 struct Student { int id; char* nume; float avg; }; int functiaStudent(struct Student* h, int size, char* studName) { char found = 0; for (i = 0; i < size && !found; i++) { if ((h[i].nume != 0) && (strcmp(h[i].nume, studName) == 0)) { found = 1; } } if (found == 1) return i; else return -1; } struct Student* HTable; int size = ARRAY_SIZE; HTable = (struct Student*)malloc(size * sizeof(struct Student)); for (int i = 0; i < size; i++) { HTable[i].nume = NULL; }

- A. **Functia implementeaza corect operatia de cautare pentru parametrul studName daca acesta este cheie pentru tabela de dispersie.** ✓correct
- B. Functia implementeaza incorect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- C. Parametrii functiei insertStudent(struct Student*, int, struct Student) sunt declarati eronat in raport cu modul lor de utilizare.
- D. Functia implementeaza corect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- E. Functia identifica toate elementele de tip Student stocate in tabela de dispersie.

Question 134 of 339

Care afirmatie este adevarata cu privire la variabila pointer p la momentul dezalocarii memoriei Heap prin apelarea functiei free(p)?

- A. **Isi pastreaza continutul, respectiv adresa de memorie Heap obtinuta la momentul alocarii.** ✓correct
- B. Se dezaloca variabila p de pe segmentul de Stack unde a fost alocata la momentul definirii sale.
- C. Se copiaza datele in memorie Stack unde vor fi accesate prin variabila pointer p.
- D. Se reinitializeaza cu pointerul nul.
- E. Se rescrie cu o adresa de memorie Heap diferita.

Question 135 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o structura de tip lista simpla?

- A. **Lista simpla este o structura liniara.** ✓correct

- B. Lista simpla este o structura dinamica in sensul ca nodurile pot fi adaugate si/sau eliminate oricand fara a se cunoaste apriori numarul acestora. ✓correct**
- C. Lista simpla contine noduri unde doua noduri diferite sunt eterogene din punct de vedere al datelor stocate.
- D. Lista simpla este o structura a carei elemente sunt referite in acces secvential. ✓correct**
- E. Lista simpla contine noduri unde doua noduri diferite sunt omogene din punct de vedere al datelor stocate. ✓correct**
- F. Lista simpla contine noduri, iar fiecare nod stocheaza mai multe elemente de acelasi tip.

Question 136 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: `#include <stdio.h> #include <malloc.h> int main() { int v[] = {10, 20}; int *pv = v; printf("v[0] = %d, pv[0] = %d\n", v[0], pv[0]); return 0; }`

- A. v[0] = 10, pv[0] = 20
- B. v[0] = 10, pv[0] = 0
- C. Aplicatia determina eroare de compilare deoarece operatorul de indexare [] nu se aplica variabilei de tip pointer.
- D. Aplicatia determina eroare de compilare deoarece adresa unei variabile vector nu poate fi atribuita unei variabile de tip pointer.
- E. v[0] = 10, pv[0] = 10 ✓correct**

Question 137 of 339

Identificati problema secventei de cod sursa urmatoare: Code: `int* p; *p = 10;`

- A. Variabila p este alocata in segmentul de cod (read-only), deci nu poate fi modificata.
- B. Pointerul este constant.
- C. Memoria nu este alocata inainte de utilizare. ✓correct**
- D. Variabila p contine adresa nula.
- E. Nu se poate modifica valoarea 10.

Question 138 of 339

Ce tipuri de rotatii se aplica pentru reechilibrarea unui arbore AVL? Selectati unul sau mai multe:

- A. Rotatie stanga-dreapta ✓correct**
- B. Rotatie stanga ✓correct**
- C. Rotatie dreapta-stanga ✓correct**
- D. Rotatie dreapta ✓correct**

Question 139 of 339

Care din valorile de mai jos poate fi cheie pentru nodul radacina dupa stergerea nodului cu cheia 55 din arborele binar de cautare a carui reprezentare pe niveluri este 55-33-77-11-44-66-88?

- A. 44 ✓correct
- B. 66 ✓correct
- C. 77
- D. 88

Question 140 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o variabila de tip pointer in limbajul C?

- A. Poate fi utilizata in orice calcul aritmetic.
- B. Prezinta o sintaxa dedicata pentru declarare. ✓correct
- C. Stocheaza o adresa de memorie. ✓correct
- D. Poate fi utilizata doar pentru accesul la segmente de date.
- E. Poate fi utilizata doar pentru accesul la memorie heap.

Question 141 of 339

Considerandu-se urmatoarea secventa, care din raspunsurile prezentate adauga un nou nod cu valoarea 4 la sfarsitul listei: Code: // Existing list: 1 -> 2 -> 3 struct Node* current = head; while (current->next != NULL) { current = current->next; } // Add code here to append a new node with value 4

- A. current->info = 4;
- B. current->next->next = malloc(sizeof(Node)); current->next->next->info = 4;
- C. current->next = malloc(sizeof(Node)); current->next->info = 4; current->next->next = NULL; ✓correct
- D. current = malloc(sizeof(Node)); current->info = 4;
- E. current->next = malloc(sizeof(Node)); current = current->next; current->info = 4; current->next = NULL; ✓correct

Question 142 of 339

In ce domenii se folosesc arborii ca structuri de date aplicate? Selectati unul sau mai multe:

- A. Definirea structurii arborescente a unui site web ✓correct
- B. Organizarea fişierelor in cadrul unui sistem de operare ✓correct
- C. Organigrame sau structuri ierarhice in companii ✓correct
- D. Implementarea căutărilor secvenţiale

Question 143 of 339

Ce presupune mecanismul de tip chaining intr-o tabela de dispersie?

- A. Folosirea unui vector auxiliar.
- B. Calcularea unei functii de dispersie.
- C. Tratarea coliziunilor printr-o lista de elemente. ✓correct**
- D. Duplicarea seturilor de date in memorie.
- E. Translatarea in coduri ASCII a valorii hash.

Question 144 of 339

Ce structura de date se utilizeaza pentru implementarea mecanismului chaining pentru o tabela de dispersie?

- A. Structura arborescenta.
- B. Structura Heap.
- C. Matrice.
- D. Lista simplu inlantuita. ✓correct**
- E. Vector.

Question 145 of 339

Care este rolul functiei hash intr-o tabela de dispersie?

- A. Sa comprime datele stocate in tabela.
- B. Sa cripteze datele stocate in tabela.
- C. Sa converteasca datele stocate in tabela in valori numerice.
- D. Sa mapeze o valoare de intrare la o pozitie in tabela. ✓correct**
- E. Sa elimine duplicatele de date stocate in tabela.

Question 146 of 339

When implementing a hash table with separate chaining, which statement is true?

- A. Each key is stored directly at the index computed by the hash function.
- B. Collisions are resolved by probing the next available slot in the array.
- C. The hash table can be implemented without an array, using only linked lists.
- D. Each array slot contains a linked list of all keys that hash to that slot. ✓correct**

Question 147 of 339

Which of the following best describes an AVL tree?

- A. A binary search tree where the height of left and right subtrees of any node differs by at most 1. ✓correct**
- B. A binary tree where every level is completely filled except possibly the last.
- C. A tree where the depth of the left subtree is always greater than the right subtree.
- D. A self-organizing search tree that uses splay operations on every access.
- E. A binary tree where the in-order traversal yields a sorted sequence.

Question 148 of 339

Which of the following is true about chaining in hash tables?

- A. Chaining cannot handle more than 2 collisions per slot.
- B. Chaining uses an array of linked lists to handle collisions. ✓correct**
- C. Chaining replaces the array with a single linked list.
- D. Chaining only works for integer keys.

Question 149 of 339

In an AVL tree, what condition triggers a rebalancing operation?

- A. The height of the tree exceeds $\log_2(n)$.
- B. A node's left and right subtree heights differ by more than 1. ✓correct**
- C. The tree contains duplicate values.
- D. The in-order traversal does not produce a sorted sequence.
- E. The depth of a leaf is exactly 1.

Question 150 of 339

What happens during a right rotation in an AVL tree?

- A. The left child becomes the new root, and the previous root becomes its right child. ✓correct**
- B. The right child becomes the new root, and the previous root becomes its left child.
- C. Both children of the unbalanced node are swapped.
- D. All nodes in the subtree are shifted one level down.
- E. The root pointer is updated to point to the right subtree.

Question 151 of 339

What is the worst-case time complexity of searching in a hash table with chaining?

- A. $O(\log n)$
- B. $O(1)$
- C. $O(n)$ ✓correct**
- D. $O(n \log n)$

Question 152 of 339

Which property must a hash function satisfy to be considered good?

- A. It must produce a unique value for every input.
- B. It must always map distinct keys to the same value.
- C. It must be deterministic and distribute keys uniformly across the table. ✓correct**
- D. It must be a one-way function.
- E. It must be reversible for all inputs.

Question 153 of 339

What does the load factor of a hash table measure?

- A. The ratio of empty slots to occupied slots.
- B. The ratio of keys to the number of slots in the table. ✓correct**
- C. The total memory used by the table in bytes.
- D. The maximum number of collisions possible.

Question 154 of 339

Which of the following operations is NOT supported by a standard binary search tree in $O(\log n)$ time when balanced?

- A. Insertion
- B. Search
- C. Deletion
- D. Finding the minimum element in $O(1)$ time without any pointer modifications. ✓correct**

Question 155 of 339

Precizati care dintre urmatoarele probleme pot fi rezolvate prin utilizarea unui arbore binar de cautare echilibrat (AVL):

- A. Sortarea unui set de date in $O(n \log n)$ cu garantie asupra limitei superioare. ✓correct**
- B. Cautarea rapida a unui element intr-un set de date dinamic cu garantie asupra limitei superioare. ✓correct**
- C. Stocarea datelor intr-o maniera ce permite cautarea in $O(\log n)$ doar daca setul de date este static.
- D. Cautarea unui element intr-un set de date dinamic fara nicio garantie asupra limitei superioare a timpului de cautare.

Question 156 of 339

Precizati care dintre urmatoarele secvente C realizeaza stergerea unui nod oarecare dintr-o lista simplu inlantuita:

- A. `while (current->next != NULL) { free(current); current = current->next; }`
- B. `if (current->next == NULL) { free(current); } else { prev->next = current->next; free(current); }`
- C. `prev->next = current->next; current->next = NULL;`
- D. `if (prev == NULL) { head = current->next; } else { prev->next = current->next; } free(current);` ✓correct**
- E. `head = current->next; free(current);`

Question 157 of 339

Care este complexitatea operatiei de inserare intr-o lista simplu inlantuita, cand se cunoaste referinta la nodul predecesor?

- A. $O(n)$

- B. $O(\log n)$
- C. **$O(1)$** ✓correct
- D. $O(n \log n)$

Question 158 of 339

Care dintre urmatoarele afirmatii este adevarata cu privire la vectorii alocati dinamic in limbajul C?

- A. Un vector alocat dinamic este alocat pe stiva de memorie.
- B. Memoria pentru un vector alocat dinamic este alocata in segmentul de date (data segment).
- C. Memoria pentru un vector alocat dinamic este eliberata automat la iesirea din blocul de cod in care a fost alocat.
- D. **Memoria pentru un vector alocat dinamic este alocata in zona de memorie heap si eliberata prin apelul functiei free().** ✓correct

Question 159 of 339

Care dintre urmatoarele sunt caracteristici ale unui arbore binar de cautare echilibrat (AVL)?
Selectati unul sau mai multe:

- A. **Diferenta dintre inaltimile subarborilor stang si drept ale oricarui nod este cel mult 1.** ✓correct
- B. **Inserarea unui nod necesita operatii de reechilibrare pentru a pastra proprietatea de arbore AVL.** ✓correct
- C. Toate nodurile frunza sunt plasate pe acelasi nivel.
- D. Inaltimea arborelui este intotdeauna $O(n)$.

Question 160 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la memoria heap in limbajul C?
Selectati unul sau mai multe:

- A. **Memoria heap este o zona de memorie ce poate fi gestionata manual de programator.** ✓correct
- B. **Memoria heap este gestionata de programator prin intermediul functiilor malloc(), calloc(), realloc() si free().** ✓correct
- C. Memoria heap este eliberata automat la terminarea functiei in care a fost alocata.
- D. Memoria heap este o zona de memorie cu dimensiune fixa stabilita la momentul compilarii programului.

Question 161 of 339

Care este complexitatea operatiei de cautare intr-o lista simplu inlantuita, in cel mai rau caz?

- A. $O(\log n)$
- B. **$O(n)$** ✓correct
- C. $O(1)$
- D. $O(n \log n)$

Question 162 of 339

Precizati care dintre urmatoarele probleme pot fi rezolvate prin utilizarea unei tabele de dispersie (hash table):

- A. Stocarea datelor intr-o maniera ce permite cautarea in $O(\log n)$ doar daca setul de date este static.
- B. Cautarea rapida a unui element intr-un set de date dinamic cu garantie asupra limitei superioare.
- C. Sortarea datelor intr-o maniera ce necesita $O(n \log n)$ cu garantie asupra limitei superioare.
- D. **Cautarea unui element intr-un set de date dinamic fara nicio garantie asupra limitei superioare a timpului de cautare.** ✓correct

Question 163 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o structura de tip arbore binar de cautare (BST)? Selectati unul sau mai multe:

- A. BST este o structura liniara.
- B. **BST este o structura dinamica in sensul ca nodurile pot fi adaugate si/sau eliminate oricand fara a se cunoaste apriori numarul acestora.** ✓correct
- C. BST este o structura a carei elemente sunt referite in acces secvential.
- D. BST este o structura a carei elemente sunt referite in acces direct.

Question 164 of 339

Care dintre urmatoarele afirmatii este adevarata cu privire la pointeri in limbajul C?

- A. **Pointerii pot pointa catre orice tip de data.** ✓correct
- B. Pointerii sunt intotdeauna alocati pe stiva de memorie.
- C. Pointerii nu pot fi dezalocati.
- D. Pointerii nu pot fi comparati cu constanta NULL.

Question 165 of 339

Care este complexitatea operatiei de inserare intr-o lista dublu inlantuita, cand se cunoaste referinta la nodul predecesor?

- A. $O(n)$
- B. $O(\log n)$
- C. **$O(1)$** ✓correct
- D. $O(n \log n)$

Question 166 of 339

Care dintre urmatoarele afirmatii este adevarata cu privire la pointerul NULL in limbajul C?

- A. Pointerul NULL este un pointer valid care pointeaza catre adresa de memorie 0xFFFFFFFF.
- B. **Pointerul NULL este un pointer special care indica faptul ca pointerul nu pointeaza catre nicio adresa de memorie valida.** ✓correct
- C. Pointerul NULL este utilizat pentru a accesa memoria heap.

D. Pointerul NULL este intotdeauna alocat pe stiva de memorie.

Question 167 of 339

Precizati care dintre urmatoarele secvente C realizeaza traversarea unei liste simplu inlantuite:

- A. `while (head != NULL) { printf("%d ", head->data); head = head->next; } ✓correct`
- B. `printf("%d ", head->data);`
- C. `for (int i = 0; i < n; i++) { printf("%d ", arr[i]); }`
- D. `head = head->next;`

Question 168 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la un arbore binar de cautare echilibrat (AVL)? Selectati unul sau mai multe:

- A. **Un arbore AVL este un arbore binar de cautare in care, pentru orice nod, diferenta dintre inaltimile subarborilor stang si drept este cel mult 1.** ✓correct
- B. **Inserarea unui nod intr-un arbore AVL necesita operatii de reechilibrare pentru a pastra proprietatea de arbore AVL.** ✓correct
- C. **Un arbore AVL permite cautarea, inserarea si stergerea in timp $O(\log n)$ in cel mai rau caz.** ✓correct
- D. Un arbore AVL este intotdeauna complet (toate nivelurile sunt pline cu exceptia posibila a ultimului nivel).

Grile Set 3

44 questions

Question 169 of 339

Dacă un vector este declarat ca `char mesaj[10]` care este indexul valid pentru ultimul element al acestui vector?

- A. 9 ✓correct
- B. last
- C. mesaj +1
- D. mesaj - 1
- E. 10

Question 170 of 339

Care dintre următoarele sunt cazuri în care se folosește rotația dreapta-stânga într-un arbore AVL? Selectați unul sau mai multe:

- A. Dezechilibru la stânga cu fiu dreapta
- B. Rădăcina are gradul de echilibru -2 și fiul stânga +1
- C. Gradul de echilibru este 0
- D. Rădăcina are gradul de echilibru +2 și fiul dreapta -1 ✓correct

Question 171 of 339

Precizați rezultatul executiei aplicatiei de mai sus: Code: `#include <stdio.h> #include <malloc.h> int main() { int* pc, c, d; pc = (int*)malloc(sizeof(int)); pc[0] = -10; c = *pc + 11; d = c + 1; printf("c = %d, d = %d, *pc = %d\n", c, d, *pc); free(pc); return 0; }`

- A. c = 1, d = 2, *pc = -10 ✓correct
- B. c = 1, d = 1, *pc = -10
- C. Aplicatia determina eroare de compilare deoarece variabilele c si d nu sunt modificate corect cu valorile calculate pe baza adresei de memorie heap stocata in variabila pointer.
- D. c = -10, d = -9, *pc = -10
- E. Aplicatia determina eroare de executie deoarece nu se alocă suficient spatiu de memorie heap la apelul functiei malloc().

Question 172 of 339

Pentru ștergere eficientă a nodurilor arbitrare, cea mai potrivită structură este:

- A. Heap binar
- B. Listă dublu înlănțuită ✓correct
- C. Listă simplu înlănțuită
- D. Listă circulară simplu înlănțuită

E. Tablou static

Question 173 of 339

Completează codul pentru eliberarea corectă a spațiului ocupat de un vector alocat dinamic

Code: `int nrEl = 10; int* v = malloc(nrEl * sizeof(int)); for (int i = 0; i < nrEl; i++) { v[i] = i; } ____ (v); v = ____;`

- A. delete
- B. clear
- C. free ✓correct**
- D. clean
- E. NULL ✓correct**
- F. malloc(nrEl)

Question 174 of 339

Selectează caracteristicile vectorilor alocați în mod static:

- A. Trebuie să fie eliberați cu ajutorul funcției free()
- B. Dimensiunea vectorului poate fi stabilită la momentul execuției
- C. Dimensiunea este fixă ✓correct**
- D. Sunt alocați pe stiva de memorie a funcției în care sunt declarați ✓correct**
- E. Pot fi redimensionați cu ajutorul funcției realloc()

Question 175 of 339

Care dintre următoarele afirmații este corectă? Code: `#include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> struct Student { int id; char* nume; float medie; }; struct Nod { struct Student st; struct Nod* next; }; int calculPozitie(char* key, int size) { int temp = 0; for (unsigned int i = 0; i < strlen(key); i++) temp += key[i]; return (temp % size); } struct Student cautaHTable(struct Nod** hTab, int size, char* nume_student) { int k = calculPozitie(nume_student, size); struct Student stud; stud.nume = NULL; struct Nod* tmp = hTab[k]; while (tmp) { if (!strcmp(tmp->st.nume, nume_student)) stud = tmp->st; tmp = tmp->next; } return stud; }`

- A. Implementarea operației de cautare este incorectă deoarece funcția hash permite accesul direct la lista de studenți care au generat conflict pe aceeași poziție.
- B. Implementarea operației de cautare este incorectă deoarece funcția hash generează valori plasate în afara zonei alocate pentru tabela de dispersie.
- C. Implementarea operației de cautare este incorectă deoarece funcția hash nu poate fi utilizată pentru nume student.
- D. Implementarea operației de cautare este corectă deoarece se utilizează exclusiv accesul secvențial la datele stocate în tabela de dispersie.
- E. Implementarea operației de cautare este corectă deoarece se utilizează funcția hash urmata de cautarea secvențială a studentului. ✓correct**

Question 176 of 339

Identificati problema secventei de cod sursa urmatoare: Code: `int* p; *p = 10;`

- A. Variabila p este alocata in segmentul de cod (read-only), deci nu poate fi modificata.
- B. Pointerul este constant.
- C. Memoria nu este alocata inainte de utilizare. ✓ correct**
- D. Variabila p contine adresa nula.
- E. Nu se poate modifica valoarea 10.

Question 177 of 339

Care dintre urmatoarele afirmatii este adevarata? Code: `#include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> #define ARRAY_SIZE 100 struct Student { int id; char* nume; float avg; }; int functiaStudent(struct Student* h, int size, char* studName) { char found = 0, i; for (i = 0; i < size && !found; i++) { if ((h[i].nume != 0) && (strcmp(h[i].nume, studName) == 0)) { found = 1; } } if (found == 1) return i; else return -1; } struct Student* HTable; int size = ARRAY_SIZE; HTable = (struct Student*)malloc(size * sizeof(struct Student)); for (int i = 0; i < size; i++) { HTable[i].nume = NULL; }`

- A. Functia implementeaza corect operatia de cautare pentru parametrul studName daca acesta este cheie pentru tabela de dispersie. ✓ correct**
- B. Functia implementeaza incorect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- C. Parametrii functiei insertStudent(struct Student*, int, struct Student) sunt declarati eronat in raport cu modul lor de utilizare.
- D. Functia implementeaza corect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- E. Functia identifica toate elementele de tip Student stocate in tabela de dispersie.

Question 178 of 339

Care afirmatie este adevarata cu privire la variabila pointer p la momentul dezaslocarii memoriei Heap prin apelarea functiei free(p)?

- A. Isi pastreaza continutul, respectiv adresa de memorie Heap obtinuta la momentul alocarii. ✓ correct**
- B. Se dezaloca variabila p de pe segmentul de Stack unde a fost alocata la momentul definirii sale.
- C. Se copiaza datele in memorie Stack unde vor fi accesate prin variabila pointer p.
- D. Se reinitializeaza cu pointerul nul.
- E. Se rescrie cu o adresa de memorie Heap diferita.

Question 179 of 339

Avantaje ale listelor înlănțuite față de heap-uri:

- A. Cea mai mare eficiență în sortare

B. Management dinamic simplificat în anumite contexte ✓correct

C. Acces aleator mai rapid

D. Inserții/ștergeri mai simple în poziții cunoscute ✓correct

E. Acces direct la elementul min / max

Question 180 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: #include <stdio.h> #include <malloc.h> int main() { int v[] = {10, 20}; int *pv = v; printf("v[0] = %d, pv[0] = %d\n", v[0], pv[0]); return 0; }

A. v[0] = 10, pv[0] = 20

B. v[0] = 10, pv[0] = 0

C. Aplicatia determina eroare de compilare deoarece operatorul de indexare [] nu se aplica variabilei de tip pointer.

D. Aplicatia determina eroare de compilare deoarece adresa unei variabile vector nu poate fi atribuita unei variabile de tip pointer.

E. v[0] = 10, pv[0] = 10 ✓correct

Question 181 of 339

In ce domenii se folosesc arborii ca structuri de date aplicate? Selectati unul sau mai multe:

A. Definirea structurii arborescente a unui site web ✓correct

B. Organizarea fișierelor in cadrul unui sistem de operare ✓correct

C. Organigrame sau structuri ierarhice in companii ✓correct

D. Implementarea căutărilor secvențiale

Question 182 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o structura de tip lista simpla?

A. Lista simpla este o structura liniara. ✓correct

B. Lista simpla este o structura dinamica in sensul ca nodurile pot fi adaugate si/sau eliminate oricand fara a se cunoaste apriori numarul acestora. ✓correct

C. Lista simpla contine noduri unde doua noduri diferite sunt eterogene din punct de vedere al datelor stocate.

D. Lista simpla este o structura a carei elemente sunt referite in acces secvential. ✓correct

E. Lista simpla contine noduri unde doua noduri diferite sunt omogene din punct de vedere al datelor stocate. ✓correct

F. Lista simpla contine noduri, iar fiecare nod stocheaza mai multe elemente de acelasi tip.

Question 183 of 339

Care din valorile de mai jos poate fi cheie pentru nodul radacina dupa stergerea nodului cu cheia 55 din arborele binar de cautare a carui reprezentare pe niveluri este 55-33-77-11-44-66-88?

A. 44 ✓correct

B. 66 ✓correct

C. 77

D. 88

Question 184 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o variabila de tip pointer in limbajul C?

A. Poate fi utilizata in orice calcul aritmetic.

B. Prezinta o sintaxa dedicata pentru declarare. ✓correct

C. Stocheaza o adresa de memorie. ✓correct

D. Poate fi utilizata doar pentru accesul la segmente de date.

E. Poate fi utilizata doar pentru accesul la memorie heap.

Question 185 of 339

Considerandu-se urmatoarea secventa, care din raspunsurile prezentate adauga un nou nod cu valoarea 4 la sfarsitul listei: Code: // Existing list: 1 -> 2 -> 3 struct Node* current = head; while (current->next != NULL) { current = current->next; } // Add code here to append a new node with value 4

A. current->info = 4;

B. current->next->next = malloc(sizeof(Node)); current->next->next->info = 4;

C. current->next = malloc(sizeof(Node)); current->next->info = 4; current->next->next = NULL; ✓correct

D. current = malloc(sizeof(Node)); current->info = 4;

E. current->next = malloc(sizeof(Node)); current = current->next; current->info = 4; current->next = NULL; ✓correct

Question 186 of 339

Ce tipuri de rotatii se aplica pentru reechilibrarea unui arbore AVL? Selectati unul sau mai multe:

A. Rotatie stanga-dreapta ✓correct

B. Rotatie stanga ✓correct

C. Rotatie dreapta-stanga ✓correct

D. Rotatie dreapta ✓correct

Question 187 of 339

Ce presupune mecanismul de tip chaining intr-o tabela de dispersie?

A. Folosirea unui vector auxiliar.

B. Calcularea unei functii de dispersie.

C. Tratarea coliziunilor printr-o lista de elemente. ✓correct

- D. Duplicarea seturilor de date in memorie.
- E. Translatarea in coduri ASCII a valorii hash.

Question 188 of 339

Ce structura de date se utilizeaza pentru implementarea mecanismului chaining pentru o tabela de dispersie?

- A. Structura arborescenta.
- B. Structura Heap.
- C. Matrice.
- D. Lista simplu inlantuita. ✓correct**
- E. Vector.

Question 189 of 339

Care este rolul functiei hash intr-o tabela de dispersie?

- A. Sa comprime datele stocate in tabela.
- B. Sa creeze datele stocate in tabela.
- C. Sa converteasca datele stocate in tabela in valori numerice.
- D. Sa mapeze o valoare de intrare la o pozitie in tabela. ✓correct**
- E. Sa elimine duplicatele de date stocate in tabela.

Question 190 of 339

In an AVL tree, what condition triggers a rebalancing operation?

- A. The height of the tree exceeds $\log_2(n)$.
- B. A node's left and right subtree heights differ by more than 1. ✓correct**
- C. The tree contains duplicate values.
- D. The in-order traversal does not produce a sorted sequence.
- E. The depth of a leaf is exactly 1.

Question 191 of 339

Which of the following best describes an AVL tree?

- A. A binary search tree where the height of left and right subtrees of any node differs by at most 1. ✓correct**
- B. A binary tree where every level is completely filled except possibly the last.
- C. A tree where the depth of the left subtree is always greater than the right subtree.
- D. A self-organizing search tree that uses splay operations on every access.
- E. A binary tree where the in-order traversal yields a sorted sequence.

Question 192 of 339

Which of the following is true about chaining in hash tables?

- A. Chaining cannot handle more than 2 collisions per slot.
- B. Chaining uses an array of linked lists to handle collisions. ✓correct**
- C. Chaining replaces the array with a single linked list.
- D. Chaining only works for integer keys.

Question 193 of 339

When implementing a hash table with separate chaining, which statement is true?

- A. Each key is stored directly at the index computed by the hash function.
- B. Collisions are resolved by probing the next available slot in the array.
- C. The hash table can be implemented without an array, using only linked lists.
- D. Each array slot contains a linked list of all keys that hash to that slot. ✓correct**

Question 194 of 339

What happens during a right rotation in an AVL tree?

- A. The left child becomes the new root, and the previous root becomes its right child. ✓correct**
- B. The right child becomes the new root, and the previous root becomes its left child.
- C. Both children of the unbalanced node are swapped.
- D. All nodes in the subtree are shifted one level down.
- E. The root pointer is updated to point to the right subtree.

Question 195 of 339

What is the worst-case time complexity of searching in a hash table with chaining?

- A. $O(\log n)$
- B. $O(1)$
- C. $O(n)$ ✓correct**
- D. $O(n \log n)$

Question 196 of 339

Which property must a hash function satisfy to be considered good?

- A. It must produce a unique value for every input.
- B. It must always map distinct keys to the same value.
- C. It must be deterministic and distribute keys uniformly across the table. ✓correct**
- D. It must be a one-way function.
- E. It must be reversible for all inputs.

Question 197 of 339

What does the load factor of a hash table measure?

- A. The ratio of empty slots to occupied slots.
- B. The ratio of keys to the number of slots in the table. ✓correct**

- C. The total memory used by the table in bytes.
- D. The maximum number of collisions possible.

Question 198 of 339

Which of the following operations is NOT supported by a standard binary search tree in $O(\log n)$ time when balanced?

- A. Insertion
- B. Search
- C. Deletion
- D. Finding the minimum element in $O(1)$ time without any pointer modifications. ✓correct

Question 199 of 339

Precizati care dintre urmatoarele probleme pot fi rezolvate prin utilizarea unui arbore binar de cautare echilibrat (AVL):

- A. Sortarea unui set de date in $O(n \log n)$ cu garantie asupra limitei superioare. ✓correct
- B. Cautarea rapida a unui element intr-un set de date dinamic cu garantie asupra limitei superioare. ✓correct
- C. Stocarea datelor intr-o maniera ce permite cautarea in $O(\log n)$ doar daca setul de date este static.
- D. Cautarea unui element intr-un set de date dinamic fara nicio garantie asupra limitei superioare a timpului de cautare.

Question 200 of 339

Care dintre urmatoarele sunt caracteristici ale unui arbore binar de cautare echilibrat (AVL)? Selectati unul sau mai multe:

- A. Diferenta dintre inaltimele subarborilor stang si drept ale oricarui nod este cel mult 1. ✓correct
- B. Inserarea unui nod necesita operatii de reechilibrare pentru a pastra proprietatea de arbore AVL. ✓correct
- C. Toate nodurile frunza sunt plasate pe acelasi nivel.
- D. Inaltimea arborelui este intotdeauna $O(n)$.

Question 201 of 339

Care este complexitatea operatiei de inserare intr-o lista simplu inlantuita, cand se cunoaste referinta la nodul predecesor?

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$ ✓correct
- D. $O(n \log n)$

Question 202 of 339

Care dintre urmatoarele afirmatii este adevarata cu privire la vectorii alocati dinamic in limbajul C?

- A. Un vector alocat dinamic este alocat pe stiva de memorie.
- B. Memoria pentru un vector alocat dinamic este alocata in segmentul de date (data segment).
- C. Memoria pentru un vector alocat dinamic este eliberata automat la iesirea din blocul de cod in care a fost alocat.
- D. Memoria pentru un vector alocat dinamic este alocata in zona de memorie heap si eliberata prin apelul functiei free(). ✓correct**

Question 203 of 339

Precizati care dintre urmatoarele secvente C realizeaza stergerea unui nod oarecare dintr-o lista simplu inlantuita:

- A. `while (current->next != NULL) { free(current); current = current->next; }`
- B. `if (current->next == NULL) { free(current); } else { prev->next = current->next; free(current); }`
- C. `prev->next = current->next; current->next = NULL;`
- D. `if (prev == NULL) { head = current->next; } else { prev->next = current->next; } free(current);` ✓correct**
- E. `head = current->next; free(current);`

Question 204 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la memoria heap in limbajul C? Selectati unul sau mai multe:

- A. Memoria heap este o zona de memorie ce poate fi gestionata manual de programator. ✓correct**
- B. Memoria heap este gestionata de programator prin intermediul functiilor malloc(), calloc(), realloc() si free(). ✓correct**
- C. Memoria heap este eliberata automat la terminarea functiei in care a fost alocata.
- D. Memoria heap este o zona de memorie cu dimensiune fixa stabilita la momentul compilarii programului.

Question 205 of 339

Care este complexitatea operatiei de cautare intr-o lista simplu inlantuita, in cel mai rau caz?

- A. $O(\log n)$
- B. $O(n)$ ✓correct**
- C. $O(1)$
- D. $O(n \log n)$

Question 206 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la un arbore binar de cautare echilibrat (AVL)? Selectati unul sau mai multe:

- A. Un arbore AVL este un arbore binar de cautare in care, pentru orice nod, diferenta dintre inaltimele subarborilor stang si drept este cel mult 1. ✓correct
- B. Inserarea unui nod intr-un arbore AVL necesita operatii de reechilibrare pentru a pastra proprietatea de arbore AVL. ✓correct
- C. Un arbore AVL permite cautarea, inserarea si stergerea in timp $O(\log n)$ in cel mai rau caz. ✓correct
- D. Un arbore AVL este intotdeauna complet (toate nivelurile sunt pline cu exceptia posibila a ultimului nivel).

Question 207 of 339

Precizati care dintre urmatoarele probleme pot fi rezolvate prin utilizarea unei tabele de dispersie (hash table):

- A. Stocarea datelor intr-o maniera ce permite cautarea in $O(\log n)$ doar daca setul de date este static.
- B. Cautarea rapida a unui element intr-un set de date dinamic cu garantie asupra limitei superioare.
- C. Sortarea datelor intr-o maniera ce necesita $O(n \log n)$ cu garantie asupra limitei superioare.
- D. Cautarea unui element intr-un set de date dinamic fara nicio garantie asupra limitei superioare a timpului de cautare. ✓correct

Question 208 of 339

Care dintre urmatoarele afirmatii este adevarata cu privire la pointeri in limbajul C?

- A. Pointerii pot pointa catre orice tip de data. ✓correct
- B. Pointerii sunt intotdeauna alocati pe stiva de memorie.
- C. Pointerii nu pot fi dezalocati.
- D. Pointerii nu pot fi comparati cu constanta NULL.

Question 209 of 339

Care este complexitatea operatiei de inserare intr-o lista dublu inlantuita, cand se cunoaste referinta la nodul predecesor?

- A. $O(n)$
- B. $O(\log n)$
- C. **$O(1)$** ✓correct
- D. $O(n \log n)$

Question 210 of 339

Care dintre urmatoarele afirmatii este adevarata cu privire la pointerul NULL in limbajul C?

- A. Pointerul NULL este un pointer valid care pointeaza catre adresa de memorie 0xFFFFFFFF.
- B. Pointerul NULL este un pointer special care indica faptul ca pointerul nu pointeaza catre nicio adresa de memorie valida. ✓correct
- C. Pointerul NULL este utilizat pentru a accesa memoria heap.
- D. Pointerul NULL este intotdeauna alocat pe stiva de memorie.

Question 211 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o structura de tip arbore binar de cautare (BST)? Selectati unul sau mai multe:

- A. BST este o structura liniara.
- B. BST este o structura dinamica in sensul ca nodurile pot fi adaugate si/sau eliminate oricand fara a se cunoaste apriori numarul acestora. ✓ correct**
- C. BST este o structura a carei elemente sunt referite in acces secvential.
- D. BST este o structura a carei elemente sunt referite in acces direct.

Question 212 of 339

Precizati care dintre urmatoarele secvente C realizeaza traversarea unei liste simplu inlantuite:

- A. `while (head != NULL) { printf("%d ", head->data); head = head->next; } ✓ correct`**
- B. `printf("%d ", head->data);`
- C. `for (int i = 0; i < n; i++) { printf("%d ", arr[i]); }`
- D. `head = head->next;`

Grile Set 4

47 questions

Question 213 of 339

Pentru ștergere eficientă a nodurilor arbitrare, cea mai potrivită structură este:

- A. Heap binar
- B. Listă dublu înlănțuită ✓correct
- C. Listă simplu înlănțuită
- D. Listă circulară simplu înlănțuită
- E. Tablou static

Question 214 of 339

Dacă un vector este declarat ca `char mesaj[10]` care este indexul valid pentru ultimul element al acestui vector?

- A. 9 ✓correct
- B. last
- C. mesaj + 1
- D. mesaj - 1
- E. 10

Question 215 of 339

Completează codul pentru eliberarea corectă spațiului ocupat de un vector alocat dinamic:

```
Code: int nrEl = 10; int* v = malloc(nrEl * sizeof(int)); for (int i = 0; i < nrEl; i++) { v[i] = i; }  
____(v); v = ____;
```

- A. delete
- B. clear
- C. free ✓correct
- D. clean
- E. NULL ✓correct

Question 216 of 339

Care dintre următoarele sunt cazuri în care se folosește rotația dreapta-stânga într-un arbore AVL?

- A. Dezechilibru la stânga cu fiu drept
- B. Rădăcina are gradul de echilibru -2 și fiul stânga +1
- C. Gradul de echilibru este 0
- D. Rădăcina are gradul de echilibru +2 și fiul dreapta -1 ✓correct
- E. (none)

Question 217 of 339

Selectează caracteristicile vectorilor alocați în mod static:

- A. Trebuie să fie eliberați cu ajutorul funcției free()
- B. Dimensiunea vectorului poate fi stabilită la momentul execuției
- C. Dimensiunea este fixă ✓correct**
- D. Sunt alocați pe stiva de memorie a funcției în care sunt declarați ✓correct**
- E. Pot fi redimensionați cu ajutorul funcției realloc()

Question 218 of 339

Care dintre următoarele afirmatii este corecta? Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> struct Student { int id; char* nume; float medie; }; struct Nod { struct Student st; struct Nod* next; }; int calculPozitie(char* key, int size) { int temp = 0; for (unsigned int i = 0; i < strlen(key); i++) temp += key[i]; return (temp % size); } struct Student cautaHTable(struct Nod** hTab, int size, char* nume_student) { int k = calculPozitie(nume_student, size); struct Student stud; stud.nume = NULL; struct Nod* tmp = hTab[k]; while (tmp) { if (!strcmp(tmp->st.nume, nume_student)) stud = tmp->st; tmp = tmp->next; } return stud; }

- A. Implementarea operatiei de cautare este incorecta deoarece functia hash permite accesul direct la lista de studenti care au generat conflict pe aceeasi pozitie.
- B. Implementarea operatiei de cautare este incorecta deoarece functia hash genereaza valori plasate in afara zonei alocate pentru tabela de dispersie.
- C. Implementarea operatiei de cautare este incorecta deoarece functia hash nu poate fi utilizata pentru nume student.
- D. Implementarea operatiei de cautare este corecta deoarece se utilizeaza exclusiv accesul secvential la datele stocate in tabela de dispersie.
- E. Implementarea operatiei de cautare este corecta deoarece se utilizeaza functia hash urmata de cautarea secventiala a studentului. ✓correct**

Question 219 of 339

Avantaje ale listelor înlănțuite față de heap-uri:

- A. Cea mai mare eficiență în sortare
- B. Management dinamic simplificat în anumite contexte ✓correct**
- C. Acces aleator mai rapid
- D. Inserții/ștergeri mai simple în poziții cunoscute ✓correct**
- E. Acces direct la elementul min / max

Question 220 of 339

Identificati problema secventei de cod sursa urmatoare: Code: int* p; *p = 10;

- A. Variabila p este alocata in segmentul de cod (read-only), deci nu poate fi modificata.
- B. Pointerul este constant.

C. Memoria nu este alocata inainte de utilizare. ✓correct

D. Variabila p contine adresa nula.

E. Nu se poate modifica valoarea 10.

Question 221 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: #include <stdio.h> #include <malloc.h> int main() { int* pc, c, d; pc = (int*)malloc(sizeof(int)); pc[0] = -10; c = *pc + 11; d = c + 1; printf("c = %d, d = %d, *pc = %d\n", c, d, *pc); free(pc); return 0; }

A. c = 1, d = 2, *pc = -10 ✓correct

B. c = 1, d = 1, *pc = -10

C. Aplicatia determina eroare de compilare deoarece variabilele c si d nu sunt modificate corect cu valorile calculate pe baza adresei de memorie heap stocata in variabila pointer

D. c = -10, d = -9, *pc = -10

E. Aplicatia determina eroare de executie deoarece nu se aloca suficient spatiu de memorie heap la apelul functiei malloc().

Question 222 of 339

Care afirmatie este adevarata cu privire la variabila pointer p la momentul dezalocarii memoriei Heap prin apelarea functiei free(p)?

A. Isi pastreaza continutul, respectiv adresa de memorie Heap obtinuta la momentul alocarii. ✓correct

B. Se dezaloca variabila p de pe segmentul de Stack unde a fost alocata la momentul definirii sale.

C. Se copiaza datele in memorie Stack unde vor fi accesate prin variabila pointer p.

D. Se reinitializeaza cu pointerul nul.

E. Se rescrie cu o adresa de memorie Heap diferita.

Question 223 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o structura de tip lista simpla?

A. Lista simpla este o structura liniara. ✓correct

B. Lista simpla este o structura dinamica in sensul ca nodurile pot fi adaugate si/sau eliminate oricand fara a se cunoaste apriori numarul acestora. ✓correct

C. Lista simpla contine noduri unde doua noduri diferite sunt eterogene din punct de vedere al datelor stocate.

D. Lista simpla este o structura a carei elemente sunt referite in acces secvential. ✓correct

E. Lista simpla contine noduri unde doua noduri diferite sunt omogene din punct de vedere al datelor stocate. ✓correct

F. Lista simpla contine noduri, iar fiecare nod stocheaza mai multe elemente de acelasi tip.

Question 224 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: `#include <stdio.h> #include <malloc.h> int main() { int v[] = {10, 20}; int *pv = v; printf("v[0] = %d, pv[0] = %d\n", v[0], pv[0]); return 0; }`

- A. `v[0] = 10, pv[0] = 20`
- B. `v[0] = 10, pv[0] = 0`
- C. Aplicatia determina eroare de compilare deoarece operatorul de indexare `[]` nu se aplica variabilei de tip pointer.
- D. Aplicatia determina eroare de compilare deoarece adresa unei variabile vector nu poate fi atribuita unei variabile de tip pointer.
- E. **`v[0] = 10, pv[0] = 10` ✓correct**

Question 225 of 339

În ce domenii se folosesc arborii ca structuri de date aplicate?

- A. **Definirea structurii arborescente a unui site web ✓correct**
- B. **Organizarea fişierelor în cadrul unui sistem de operare ✓correct**
- C. **Organigrame sau structuri ierarhice în companii ✓correct**
- D. Implementarea căutărilor secvenţiale

Question 226 of 339

Ce tipuri de rotaţii se aplică pentru reechilibrarea unui arbore AVL?

- A. **Rotaţie stânga-dreapta ✓correct**
- B. **Rotaţie stânga ✓correct**
- C. **Rotaţie dreapta-stânga ✓correct**
- D. **Rotaţie dreapta ✓correct**

Question 227 of 339

Care din valorile de mai jos poate fi cheia pentru nodul radacina după ştergerea nodului cu cheia 55 din arborele binar de cautare a carui reprezentare pe niveluri este 55-33-77-11-44-66-88?

- A. **44 ✓correct**
- B. **66 ✓correct**
- C. 77
- D. 88

Question 228 of 339

Care dintre următoarele afirmaţii sunt adevărate cu privire la o variabilă de tip pointer în limbajul C?

- A. Poate fi utilizată în orice calcul aritmetic.
- B. **Prezintă o sintaxă dedicată pentru declarare. ✓correct**

C. Stochează o adresă de memorie. ✓correct

D. Poate fi utilizată doar pentru accesul la segmente de date.

E. Poate fi utilizată doar pentru accesul la memorie heap.

Question 229 of 339

Considerându-se următoarea secvență, care dintre fragmentele procedurale adaugă un nou nod la sfârșitul listei? Code: // Existing list: 9 -> 2 -> 3 struct Node* current = head; while (current->next != NULL) { current = current->next; } // Add code here to append a new node with value 4

A. current->info = 4;

B. current->next->next = malloc(sizeof(Node)); current->next->next->info = 4;

C. current->next = malloc(sizeof(Node)); current->next->info = 4; current->next->next = NULL; ✓correct

D. current = malloc(sizeof(Node)); current->info = 4;

E. current->next = malloc(sizeof(Node)); current = current->next; current->info = 4; current->next = NULL; ✓correct

Question 230 of 339

Care dintre următoarele afirmatii este adevarata? Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> #define ARRAY_SIZE 100 struct Student { int id; char* nume; float avg; }; int functiaStudent(struct Student* h, int size, char* studName) { char found = 0, i; for (i = 0; i < size && !found; i++) { if ((h[i].nume != 0) && (strcmp(h[i].nume, studName) == 0)) { found = 1; } } if (found == 1) return i; else return -1; } struct Student* HTable; int size = ARRAY_SIZE; HTable = (struct Student*)malloc(size * sizeof(struct Student)); for (int i = 0; i < size; i++) { HTable[i].nume = NULL; }

A. Functia implementeaza corect operatia de cautare pentru parametrul studName daca acesta este cheie pentru tabela de dispersie. ✓correct

B. Functia implementeaza incorect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.

C. Parametrii functiei insertStudent(struct Student*, int, struct Student) sunt declarati eronat in raport cu modul lor de utilizare.

D. Functia implementeaza corect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.

E. Functia identifica toate elementele de tip Student stocate in tabela de dispersie.

Question 231 of 339

Ce structura de date se utilizeaza pentru implementarea mecanismului chaining pentru o tabela de dispersie?

A. Structura arborescenta.

B. Structura Heap.

- C. Matrice.
- D. Lista simplu înlantuita. ✓correct**
- E. Vector.

Question 232 of 339

Ce presupune mecanismul de tip chaining într-o tabela de dispersie?

- A. Folosirea unui vector auxiliar.
- B. Calcularea unei functii de dispersie.
- C. Tratarea coliziunilor printr-o lista de elemente. ✓correct**
- D. Duplicarea seturilor de date in memorie.
- E. Translatarea in coduri ASCII a valorii hash.

Question 233 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la urmatoarea secventa de cod sursa C: Code: #include <stdio.h> #include <malloc.h> #define DIM 100 #define SPERS NULL struct Student { int id; char* name; float media; }; struct Nod { struct Student s; struct Nod* next; }; struct Nod** stTable; struct Student stKey; stTable = (struct Nod**)malloc(sizeof(struct Nod*) * DIM); for (int i = 0; i < DIM; i++) stTable[i] = 0;

- A. Initializarea elementelor din vector este efectuata cu valoarea unui pointer nul. ✓correct**
- B. Initializeaza un vector de 100 de elemente de tip Nod*. ✓correct**
- C. Aloca si initializeaza structura suport pentru o tabela de dispersie care implementeaza mecanismul de tip chaining pentru rezolvarea cazurilor de coliziune. ✓correct**
- D. Aloca un vector cu dimensiunea 100 de elemente. ✓correct**
- E. Initializeaza un vector de 100 de elemente de tip Nod**.

Question 234 of 339

Pentru ștergere eficientă a nodurilor arbitrare, cea mai potrivită structură este:

- A. Heap binar
- B. Listă dublu înlănțuită ✓correct**
- C. Listă simplu înlănțuită
- D. Listă circulară simplu înlănțuită
- E. Tablou static

Question 235 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: #include <stdio.h> #include <malloc.h> int main() { int* pc, c, d; pc = (int*)malloc(sizeof(int)); pc[0] = -10; c = *pc + 11; d = c + 1; printf("c = %d, d = %d, *pc = %d\n", c, d, *pc); free(pc); return 0; }

- A. c = 1, d = 2, *pc = -10 ✓correct**
- B. c = 1, d = 1, *pc = -10

- C. Aplicatia determina eroare de compilare deoarece variabilele c si d nu sunt modificate corect cu valorile calculate pe baza adresei de memorie heap stocata in variabila pointer
- D. c = -10, d = -9, *pc = -10
- E. Aplicatia determina eroare de executie deoarece nu se alocă suficient spatiu de memorie heap la apelul functiei malloc().

Question 236 of 339

Dacă un vector este declarat ca `char mesaj[10]` care este indexul valid pentru ultimul element al acestui vector?

- A. 9 ✓correct
- B. last
- C. mesaj + 1
- D. mesaj - 1
- E. 10

Question 237 of 339

Care este afirmația corectă cu privire la următoarea funcție? Code: `Student*`

```
functionList(Node** head) {
    Student* student = NULL;
    if (*head != NULL) {
        student = (*head)->info;
        Node* tmp = *head;
        if ((*head)->next != (*head)) {
            tmp->prev->next = tmp->next;
            tmp->next->prev = tmp->prev;
            *head = tmp->next;
        }
        else
            *head = NULL;
        free(tmp);
    }
    return student;
}
```

- A. funcția inserează un element la începutul unei liste simplu înlanțuite
- B. funcția inserează un nod la începutul unei liste dublu înlanțuite circulare
- C. capatul listei este trimis prin valoare pentru a nu-l actualiza
- D. funcția returnează informația utilă existentă în capatul listei ✓correct
- E. stergerea containerului echivalent primul nod din lista ✓correct

Question 238 of 339

Care dintre următoarele sunt cazuri în care se folosește rotația dreapta-stânga într-un arbore AVL?

- A. Dezechilibru la stânga cu fiu drept
- B. Rădăcina are gradul de echilibru -2 și fiul stânga +1
- C. Gradul de echilibru este 0
- D. Rădăcina are gradul de echilibru +2 și fiul dreapta -1 ✓correct
- E. (none)

Question 239 of 339

Selectează caracteristicile vectorilor alocați în mod static:

- A. Trebuie să fie eliberați cu ajutorul funcției `free()`
- B. Dimensiunea vectorului poate fi stabilită la momentul execuției

C. Dimensiunea este fixă ✓correct

D. Sunt alocați pe stiva de memorie a funcției în care sunt declarați ✓correct

E. Pot fi redimensionați cu ajutorul funcției realloc()

Question 240 of 339

Care dintre următoarele afirmatii este corecta? Code: #include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> struct Student { int id; char* nume; float medie; }; struct Nod { struct Student st; struct Nod* next; }; int calculPozitie(char* key, int size) { int temp = 0; for (unsigned int i = 0; i < strlen(key); i++) temp += key[i]; return (temp % size); } struct Student cautareHTable(struct Nod** hTab, int size, char* nume_student) { int k = calculPozitie(nume_student, size); struct Student stud; stud.nume = NULL; struct Nod* tmp = hTab[k]; while (tmp) { if (!strcmp(tmp->st.nume, nume_student)) stud = tmp->st; tmp = tmp->next; } return stud; }

A. Implementarea operatiei de cautare este incorecta deoarece functia hash permite accesul direct la lista de studenti care au generat conflict pe aceeasi pozitie.

B. Implementarea operatiei de cautare este incorecta deoarece functia hash genereaza valori plasate in afara zonei alocate pentru tabela de dispersie.

C. Implementarea operatiei de cautare este incorecta deoarece functia hash nu poate fi utilizata pentru nume student.

D. Implementarea operatiei de cautare este corecta deoarece se utilizeaza exclusiv accesul secvential la datele stocate in tabela de dispersie.

E. Implementarea operatiei de cautare este corecta deoarece se utilizeaza functia hash urmata de cautarea secventiala a studentului. ✓correct

Question 241 of 339

Ce funcție este folosită pentru a aloca memorie in mod dinamic pentru un vector de elemente in limbajul C?

A. malloc() ✓correct

B. printf()

C. sizeof()

D. strlen()

E. scanf()

Question 242 of 339

Avantaje ale listelor înlănțuite față de heap-uri:

A. Cea mai mare eficiență în sortare

B. Management dinamic simplificat în anumite contexte ✓correct

C. Acces aleator mai rapid

D. Inserții/ștergeri mai simple în poziții cunoscute ✓correct

E. Acces direct la elementul min / max

Question 243 of 339

Identificati problema secventei de cod sursa urmatoare: Code: `int* p; *p = 10;`

- A. Variabila p este alocata in segmentul de cod (read-only), deci nu poate fi modificata.
- B. Pointerul este constant.
- C. Memoria nu este alocata inainte de utilizare. ✓ correct**
- D. Variabila p contine adresa nula.
- E. Nu se poate modifica valoarea 10.

Question 244 of 339

Care dintre urmatoarele afirmatii este adevarata? Code: `#include <stdio.h> #include <malloc.h> #include <string.h> #include <stdlib.h> #define ARRAY_SIZE 100 struct Student { int id; char* nume; float avg; }; int functiaStudent(struct Student* h, int size, char* studName) { char found = 0, i; for (i = 0; i < size && !found; i++) { if ((h[i].nume != 0) && (strcmp(h[i].nume, studName) == 0)) { found = 1; } } if (found == 1) return i; else return -1; } struct Student* HTable; int size = ARRAY_SIZE; HTable = (struct Student*)malloc(size * sizeof(struct Student)); for (int i = 0; i < size; i++) { HTable[i].nume = NULL; }`

- A. Functia implementeaza corect operatia de cautare pentru parametrul studName daca acesta este cheie pentru tabela de dispersie. ✓ correct**
- B. Functia implementeaza incorect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- C. Parametrii functiei insertStudent(struct Student*, int, struct Student) sunt declarati eronat in raport cu modul lor de utilizare.
- D. Functia implementeaza corect operatia de cautare pentru parametrul studName daca nu este cheie pentru tabela de dispersie.
- E. Functia identifica toate elementele de tip Student stocate in tabela de dispersie.

Question 245 of 339

Completează codul pentru eliberarea corectă spațiului ocupat de un vector alocat dinamic: Code: `int nrEl = 10; int* v = malloc(nrEl * sizeof(int)); for (int i = 0; i < nrEl; i++) { v[i] = i; } ____ (v); v = ____;`

- A. delete
- B. clear
- C. free ✓ correct**
- D. clean
- E. NULL ✓ correct**

Question 246 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la o structura de tip lista simpla?

- A. Lista simpla este o structura liniara. ✓ correct**

- B. Lista simpla este o structura dinamica in sensul ca nodurile pot fi adaugate si/sau eliminate oricand fara a se cunoaste apriori numarul acestora. ✓correct**
- C. Lista simpla contine noduri unde doua noduri diferite sunt eterogene din punct de vedere al datelor stocate.
- D. Lista simpla este o structura a carei elemente sunt referite in acces secvential. ✓correct**
- E. Lista simpla contine noduri unde doua noduri diferite sunt omogene din punct de vedere al datelor stocate. ✓correct**
- F. Lista simpla contine noduri, iar fiecare nod stocheaza mai multe elemente de acelasi tip.

Question 247 of 339

Se considera urmatoarea secventa de cod sursa in limbajul C. Precizati rezultatul executiei aplicatiei de mai sus: Code: #include <stdio.h> #include <malloc.h> int main() { int v[] = {10, 20}; int *pv = v; printf("v[0] = %d, pv[0] = %d\n", v[0], pv[0]); return 0; }

- A. v[0] = 10, pv[0] = 20
- B. v[0] = 10, pv[0] = 0
- C. Aplicatia determina eroare de compilare deoarece operatorul de indexare [] nu se aplica variabilei de tip pointer.
- D. Aplicatia determina eroare de compilare deoarece adresa unei variabile vector nu poate fi atribuita unei variabile de tip pointer.
- E. v[0] = 10, pv[0] = 10 ✓correct**

Question 248 of 339

În ce domenii se folosesc arborii ca structuri de date aplicate?

- A. Definirea structurii arborescente a unui site web ✓correct**
- B. Organizarea fișierelor în cadrul unui sistem de operare ✓correct**
- C. Organigrame sau structuri ierarhice în companii ✓correct**
- D. Implementarea căutărilor secvențiale

Question 249 of 339

Ce tipuri de rotații se aplică pentru reechilibrarea unui arbore AVL?

- A. Rotație stânga-dreapta ✓correct**
- B. Rotație stânga ✓correct**
- C. Rotație dreapta-stânga ✓correct**
- D. Rotație dreapta ✓correct**

Question 250 of 339

Care din valorile de mai jos poate fi cheia pentru nodul radacina după ștergerea nodului cu cheia 55 din arborele binar de cautare a carui reprezentare pe niveluri este 55-33-77-11-44-66-88?

- A. 44 ✓correct**

B. 66 ✓ *correct*

C. 77

D. 88

Question 251 of 339

Care dintre următoarele afirmații sunt adevărate cu privire la o variabilă de tip pointer în limbajul C?

A. Poate fi utilizată în orice calcul aritmetic.

B. Prezintă o sintaxă dedicată pentru declarare. ✓ *correct*

C. Stociază o adresă de memorie. ✓ *correct*

D. Poate fi utilizată doar pentru accesul la segmente de date.

E. Poate fi utilizată doar pentru accesul la memorie heap.

Question 252 of 339

Considerand ca urmatoarea secventa care de mai jos reprezinta procedeul utilizat de catre nodul de valoare 4 la sfarsitul secventei curente: Code: // Existing list: 3 -> 2 -> 1 struct node* current = head; while (current->next != NULL) { current = current->next; } // Add code here to append a new node with value 4

A. current->info = 4;

B. current->next->next = malloc(sizeof(Node)); current->next->next->info = 4;

C. current->next = malloc(sizeof(Node)); current->next->info = 4; current->next->next = NULL; ✓ *correct*

D. current = malloc(sizeof(Node)); current->info = 4;

E. current->next = malloc(sizeof(Node)); current = current->next; current->info = 4; current->next = NULL; ✓ *correct*

Question 253 of 339

Care afirmatie este adevarata cu privire la variabila pointer p la momentul dealocarii memoriei Heap prin apelarea functiei free(p)?

A. Isi pastreaza continutul, respectiv adresa de memorie Heap obtinuta la momentul alocarii. ✓ *correct*

B. Se dealoca variabila p de pe segmentul de Stack unde a fost alocata la momentul definirii sale.

C. Se copiaza datele in memorie Stack unde vor fi accesate prin variabila pointer p.

D. Se reinitializeaza cu pointerul nul.

E. Se rescrie cu o adresa de memorie Heap diferita.

Question 254 of 339

Ce structura de date se utilizeaza pentru implementarea mecanismului chaining pentru o tabela de dispersie?

A. Structura arborescenta.

- B. Structura Heap.
- C. Matrice.
- D. Lista simplu inlantuita. ✓correct**
- E. Vector.

Question 255 of 339

Operatii aplicabile arborilor binari de cautare includ:

- A. Cautare nod dupa o anumita valoare a cheii ✓correct**
- B. Traversare postordine ✓correct**
- C. Sortare topologica
- D. Inserare nod nou ✓correct**
- E. N/A

Question 256 of 339

Ce structura de date se utilizeaza pentru implementarea mecanismului chaining pentru o tabela de dispersie?

- A. Structura arborescenta.
- B. Structura Heap.
- C. Matrice.
- D. Lista simplu inlantuita. ✓correct**
- E. Vector.

Question 257 of 339

Ce valoare este afişată în urma rulării secvenţei de cod alăturate? Code: `int v[5] = { 1, 2, 3, 4, 5 }; printf("%d", v[3]);`

- A. 5
- B. 2
- C. 3
- D. 4 ✓correct**
- E. 1

Question 258 of 339

Care dintre urmatoarele afirmatii sunt adevarate cu privire la urmatoarea secventa de cod sursa C: Code: `#include <stdio.h> #include <malloc.h> #define DIM 100 #define SPERS NULL struct Student { int id; char* name; float media; }; struct Nod { struct Student s; struct Nod* next; }; struct Nod** stTable; struct Student stKey; stTable = (struct Nod**)malloc(sizeof(struct Nod*) * DIM); for (int i = 0; i < DIM; i++) stTable[i] = 0;`

- A. Initializarea elementelor din vector este efectuata cu valoarea unui pointer nul. ✓correct**
- B. Initializeaza un vector de 100 de elemente de tip Nod*. ✓correct**

- C. Alocă și initializează structura suport pentru o tabelă de dispersie care implementează mecanismul de tip chaining pentru rezolvarea cazurilor de coliziune.** ✓correct
- D. Alocă un vector cu dimensiunea 100 de elemente.** ✓correct
- E. Initializează un vector de 100 de elemente de tip Nod**.

Question 259 of 339

Care este afirmația corectă cu privire la următoarea funcție? Code: Student*

```
functionList(Node** head) {
    Student* student = NULL;
    if (*head != NULL) {
        student = (*head)->info;
        Node* tmp = *head;
        if ((*head)->next != (*head)) {
            tmp->prev->next = tmp->next;
            tmp->next->prev = tmp->prev;
            *head = tmp->next;
        }
        else
            *head = NULL;
        free(tmp);
    }
    return student;
}
```

- A. funcția inserează un element la începutul unei liste simplu înlănțuite
- B. funcția inserează un nod la începutul unei liste dublu înlănțuite
- C. capatul listei este trimis prin valoare pentru a nu-l actualiza
- D. funcția returnează informația utilă existentă în capatul listei** ✓correct
- E. ștergerea containerului echivalent primul nod din listă** ✓correct

Linked Lists

3 questions

Question 260 of 339

What is a linked list node typically composed of?

- A. **Stored data plus one or more links to other nodes** ✓correct
- B. Only contiguous memory cells
- C. Only a hash function
- D. Only a comparison function

Question 261 of 339

What is the cost of searching an unsorted singly linked list by value?

- A. **$O(n)$** ✓correct
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n \log n)$

Question 262 of 339

Which linked list operation can be $O(1)$ if the target position/node is already known?

- A. **Insertion after a known node** ✓correct
- B. Searching by value
- C. Sorting the list
- D. Finding the middle without extra information

SDD Test

74 questions

Question 263 of 339

Identificați problema secvenței de cod sursă următoare: Cod: `int* p; *p = 10;`

- A. Variabila `p` este alocată în segmentul de cod (read-only), deci nu poate fi modificată.
- B. Pointerul este constant.
- C. **Memoria nu este alocată înainte de utilizare.** ✓ correct
- D. Variabila `p` conține adresa nulă.
- E. Nu se poate modifica valoarea 10.

Question 264 of 339

Care dintre următoarele afirmații sunt adevărate cu privire la o variabilă de tip pointer în limbajul C?

- A. Poate fi utilizată în orice calcul aritmetic.
- B. **Prezintă o sintaxă dedicată pentru declarare.** ✓ correct
- C. **Stochează o adresă de memorie.** ✓ correct
- D. Poate fi utilizată doar pentru accesul la segmente de date.
- E. Poate fi utilizată doar pentru accesul la memorie heap.

Question 265 of 339

Precizați rezultatul execuției aplicației de mai jos: Cod: `int v[] = {10, 20}; int *pv = v; printf("v[0] = %d, pv[0] = %d\n", v[0], pv[0]);`

- A. `v[0] = 10, pv[0] = 20`
- B. `v[0] = 10, pv[0] = 0`
- C. Eroare de compilare: operatorul `[]` nu se aplică unui pointer.
- D. Eroare de compilare: adresa unui vector nu poate fi atribuită unui pointer.
- E. **`v[0] = 10, pv[0] = 10`** ✓ correct

Question 266 of 339

Precizați rezultatul execuției aplicației de mai jos: Cod: `int* pc, c, d; pc = (int*)malloc(sizeof(int)); pc[0] = -10; c = *pc + 11; d = c + 1; printf("c = %d, d = %d, *pc = %d\n", c, d, *pc); free(pc);`

- A. **`c = 1, d = 2, *pc = -10`** ✓ correct
- B. `c = 1, d = 1, *pc = -10`
- C. Eroare de compilare: `c` și `d` nu sunt modificate corect.
- D. `c = -10, d = -9, *pc = -10`
- E. Eroare de execuție: nu se alocă suficient spațiu heap.

Question 267 of 339

Completați codul pentru eliberarea corectă a spațiului ocupat de un vector alocat dinamic (spațiul 1 / spațiul 2): Cod: `int* v = malloc(nrEl * sizeof(int)); /* ... */ [ spațiu 1 ](v); v = [ spațiu 2 ];`

- A. `free(v); v = NULL;` ✓ *correct*
- B. `delete(v); v = NULL;`
- C. `free(v); v = malloc(nrEl);`
- D. `clear(v); v = NULL;`
- E. `clean(v); v = NULL;`

Question 268 of 339

Care afirmație este adevărată cu privire la variabila pointer p la momentul dezalocării memoriei Heap prin apelarea funcției `free(p)`?

- A. **Își păstrează conținutul, respectiv adresa de memorie Heap obținută la momentul alocării.** ✓ *correct*
- B. Se dezalocă variabila p de pe segmentul de Stack unde a fost alocată.
- C. Se copiază datele în memoria Stack unde vor fi accesate prin pointerul p.
- D. Se reinițializează cu pointerul nul.
- E. Se rescrie cu o adresă de memorie Heap diferită.

Question 269 of 339

Ce valoare este afișată în urma rulării secvenței de cod? Cod: `int v[5] = { 1, 2, 3, 4, 5 }; printf("%d", v[3]);`

- A. 5
- B. 2
- C. **4** ✓ *correct*
- D. 3
- E. 1

Question 270 of 339

Dacă un vector este declarat ca `char mesaj[10]`, care este indexul valid pentru ultimul element al acestui vector?

- A. **9** ✓ *correct*
- B. last
- C. mesaj + 1
- D. mesaj - 1
- E. 10

Question 271 of 339

Selectați caracteristicile vectorilor alocați în mod static:

- A. Trebuie să fie eliberați cu ajutorul funcției free()
- B. Dimensiunea vectorului poate fi stabilită la momentul execuției
- C. Dimensiunea este fixă ✓correct**
- D. Sunt alocați pe stiva de memorie a funcției în care sunt declarați ✓correct**
- E. Pot fi redimensionați cu ajutorul funcției realloc()

Question 272 of 339

Ce presupune mecanismul de tip chaining într-o tabelă de dispersie?

- A. Folosirea unui vector auxiliar.
- B. Calcularea unei funcții de dispersie.
- C. Tratarea coliziunilor printr-o listă de elemente. ✓correct**
- D. Duplicarea seturilor de date în memorie.
- E. Translatarea în coduri ASCII a valorii hash.

Question 273 of 339

Ce structură de date se utilizează pentru implementarea mecanismului chaining pentru o tabelă de dispersie?

- A. Structura arborescentă.
- B. Structura Heap.
- C. Matrice.
- D. Listă simplu înlănțuită. ✓correct**
- E. Vector.

Question 274 of 339

Care dintre următoarele afirmații sunt adevărate cu privire la secvența de cod? Cod: struct Nod** hTable; hTable = (struct Nod**)malloc(sizeof(struct Nod*) * DIM); for (int i = 0; i < DIM; i++) hTable[i] = NULL;

- A. Inițializarea elementelor din vector este efectuată cu valoarea unui pointer nul. ✓correct**
- B. Inițializează un vector de 100 de elemente de tip Nod*. ✓correct**
- C. Alocă și inițializează structura suport pentru o tabelă de dispersie cu mecanism de tip chaining. ✓correct**
- D. Alocă un vector cu dimensiunea de 100 de elemente. ✓correct**
- E. Inițializează un vector de 100 de elemente de tip Nod.

Question 275 of 339

Tabelă de dispersie cu linear probing. Care afirmație este adevărată despre funcția de căutare? Cod: int functieStudent(struct Student* ht, int size, char* studName){ char found =

```
0, i; for (i = 0; i < size && !found; i++) if (ht[i].name != NULL && strcmp(ht[i].name,
studName)==0) found = 1; if (found == 1) return i; else return -1; }
```

- A. Implementează corect căutarea dacă studName este cheie pentru tabela de dispersie.** ✓correct
- B. Implementează incorect căutarea dacă studName nu este cheie.
- C. Parametrii funcției insertStudent(...) sunt declarați eronat.
- D. Implementează corect căutarea dacă studName nu este cheie.
- E. Identifică toate elementele de tip Student stocate în tabelă.

Question 276 of 339

Tabelă de dispersie cu chaining, având numele studentului drept cheie. Care afirmație este corectă despre funcția de căutare? Cod: struct Student cautăInTable(struct Nod** hab, int size, char* nume){ int index = calculPozitie(nume, size); struct Nod* tmp = hab[index]; while (tmp){ if (strcmp(tmp->std.nume, nume)==0){ /* ... */ } tmp = tmp->next; } /* ... */ }

- A. Incorectă: funcția hash permite accesul direct la lista de studenți care au generat conflict.
- B. Incorectă: funcția hash generează valori în afara zonei alocate.
- C. Incorectă: funcția hash nu poate fi utilizată pentru nume de student.
- D. Corectă: se utilizează exclusiv accesul secvențial la datele stocate.
- E. Corectă: se utilizează funcția hash urmată de căutarea secvențială a studentului.** ✓correct

Question 277 of 339

Care dintre următoarele afirmații sunt adevărate cu privire la o structură de tip listă simplă?

- A. Lista simplă este o structură liniară.** ✓correct
- B. Este o structură dinamică: nodurile pot fi adăugate/eliminate oricând, fără a cunoaște apriori numărul lor.** ✓correct
- C. Două noduri diferite sunt eterogene din punct de vedere al datelor stocate.
- D. Elementele sunt referite în acces secvențial.** ✓correct
- E. Două noduri diferite sunt omogene din punct de vedere al datelor stocate.** ✓correct
- F. Fiecare nod stochează mai multe elemente de același tip.

Question 278 of 339

Pentru ștergerea eficientă a nodurilor arbitrare, cea mai potrivită structură este:

- A. Heap binar
- B. Listă dublu înălțuită** ✓correct
- C. Listă simplu înălțuită
- D. Listă circulară simplu înălțuită
- E. Tablou static

Question 279 of 339

Avantaje ale listelor înlănțuite față de heap-uri:

- A. Cea mai mare eficiență în sortare
- B. Management dinamic simplificat în anumite contexte ✓correct**
- C. Acces aleator mai rapid
- D. Inserții/ștergeri mai simple în poziții cunoscute ✓correct**
- E. Acces direct la elementul min/max

Question 280 of 339

După parcurgerea listei până la ultimul nod, ce cod adaugă corect un nod nou cu valoarea 4 la finalul listei? Cod: while (current->next != NULL){ current = current->next; } /* punctul A */

- A. current->info = 4;
- B. current->next->next = malloc(sizeof(Node)); current->next->next->info = 4;
- C. current->next = malloc(sizeof(Node)); current->next->info = 4; current->next->next = NULL; ✓correct**
- D. current = malloc(sizeof(Node)); current->info = 4;
- E. current->next = malloc(sizeof(Node)); current = current->next; current->info = 4; current->next = NULL; ✓correct**

Question 281 of 339

Care este afirmația corectă cu privire la funcția de mai jos? Cod: Student functionList(Node** head){ Student s = {...}; if (*head != NULL){ s = (*head)->info; Node* tmp = *head; if ((*head)->next != (*head)){ tmp->prev->next = tmp->next; tmp->next->prev = tmp->prev; *head = tmp->next; } else { *head = NULL; } free(tmp); } return s; }

- A. Inserează un element la începutul unei liste simplu înlănțuite.
- B. Inserează un nod la începutul unei liste dublu înlănțuite circulare.
- C. Capătul listei este transmis prin valoare pentru a nu-l actualiza.
- D. Returnează informația utilă existentă în capătul listei. ✓correct**
- E. Realizează ștergerea containerului echivalent primului nod din listă. ✓correct**

Question 282 of 339

Operații aplicabile arborilor binari de căutare includ:

- A. Căutare nod după o anumită valoare a cheii ✓correct**
- B. Traversare postordine ✓correct**
- C. Sortare topologică
- D. Inserare nod nou ✓correct**

Question 283 of 339

În ce domenii se folosesc arborii ca structuri de date aplicate?

- A. Definirea structurii arborescente a unui site web ✓correct**

B. Organizarea fișierelor în cadrul unui sistem de operare ✓correct

C. Organigrame sau structuri ierarhice în companii ✓correct

D. Implementarea căutărilor secvențiale

Question 284 of 339

Care valoare poate fi cheie pentru nodul rădăcină după ștergerea nodului 55 din arborele binar de căutare cu reprezentarea pe niveluri 55-33-77-11-44-66-88?

A. 44 ✓correct

B. 66 ✓correct

C. 77

D. 88

Question 285 of 339

Ce metodă de căutare folosim pentru a găsi un nod într-un arbore?

A. Bubble Sort

B. Metoda selecției

C. Divide et Impera ✓correct

D. Metoda numărării

Question 286 of 339

Arborii binari de căutare sunt:

A. arbori oarecare

B. arbori binari ✓correct

Question 287 of 339

Având un nod X cu 2 copii și Y succesorul lui X (în ordine), care afirmație este adevărată?

A. Y nu are niciun copil în dreapta

B. Y nu are niciun copil în stânga ✓correct

C. Niciuna de mai sus

D. Y are 2 copii

Question 288 of 339

Care dintre următoarele afirmații este falsă?

A. Subarborii din stânga și dreapta ar trebui să fie tot arbori binari de căutare

B. Niciunul dintre cele de mai sus ✓correct

C. Copilul din dreapta e întotdeauna mai mare decât părintele său

D. Copilul din stânga e întotdeauna mai mic decât părintele său

Question 289 of 339

Care traversare afișează elementele unui arbore binar de căutare în ordine crescătoare (sortat)?

- A. Inordine ✓ *correct*
- B. Preordine
- C. Postordine
- D. Pe nivel

Question 290 of 339

Ce se poate afirma despre traversarea inordine a unui arbore binar de căutare?

- A. Niciun răspuns corect
- B. Traversează arborele în ordine descrescătoare
- C. Traversează arborele în ordine crescătoare ✓ *correct*
- D. Traversează arborele la întâmplare

Question 291 of 339

Înălțimea unui arbore binar de căutare construit aleator este:

- A. Distanța de la primul la ultimul nod din subarborele stâng
- B. Niciun răspuns corect
- C. $O(\log n)$ ✓ *correct*
- D. Distanța de la primul la ultimul nod

Question 292 of 339

Într-o traversare postordine, nodurile rădăcină apar la final.

- A. Adevărat ✓ *correct*
- B. Fals

Question 293 of 339

Într-un arbore binar de căutare, o nouă valoare se inserează mereu ca:

- A. frunză ✓ *correct*
- B. oriunde
- C. lângă cel mai mare element
- D. rădăcină

Question 294 of 339

Într-un arbore binar de căutare, valoarea asociată copilului din dreapta unui nod este întotdeauna:

- A. mai mare decât valoarea nodului ✓ *correct*
- B. mai mare sau egală decât valoarea nodului

- C. mai mică decât valoarea nodului
- D. mai mică sau egală decât valoarea nodului

Question 295 of 339

Parcurgerea nerecursivă a unui arbore binar se poate face folosind o:

- A. listă dublă
- B. tabelă de dispersie
- C. listă simplă
- D. **stivă** ✓correct

Question 296 of 339

Principala metodă de rezolvare a problemelor cu arbori binari este:

- A. permutări
- B. **recursivitate** ✓correct
- C. backtracking

Question 297 of 339

Timpul de căutare pentru arborii binari (în cazul cel mai defavorabil) este identic cu:

- A. **cel al unei liste simplu înlănțuite** ✓correct
- B. cel al unei tabele de dispersie
- C. cel al unei liste dublu înlănțuite

Question 298 of 339

Toți arborii binari de căutare sunt echilibrați.

- A. Adevărat
- B. **Fals** ✓correct

Question 299 of 339

Care dintre următoarele sunt cazuri în care se folosește rotația dreapta-stânga într-un arbore AVL?

- A. Dezechilibru la stânga cu fiu dreapta
- B. Rădăcina are gradul de echilibru -2 și fiul stâng +1
- C. Gradul de echilibru este 0
- D. **Rădăcina are gradul de echilibru +2 și fiul drept -1** ✓correct

Question 300 of 339

Ce tipuri de rotații se aplică pentru reechilibrarea unui arbore AVL?

- A. **Rotație stânga-dreapta** ✓correct
- B. **Rotație stânga** ✓correct

C. Rotație dreapta-stânga ✓correct

D. Rotație dreapta ✓correct

Question 301 of 339

Arborele binar este echilibrat când:

A. unul dintre factorii de echilibrare ai nodurilor este 0

B. toți factorii de echilibrare sunt egali cu suma nodurilor

C. unul dintre factorii de echilibrare este egal cu suma nodurilor

D. toți factorii de echilibrare ai nodurilor sunt -1, 0, +1 ✓correct

Question 302 of 339

Arborii AVL se mai numesc și arbori:

A. conecși

B. echilibrați ✓correct

C. sortați

D. dezechilibrați

Question 303 of 339

Arborii AVL sunt arbori binari ordonați, care au în plus o proprietate de echilibru:

A. înălțimea subarborelui stâng este egală cu înălțimea subarborelui drept

B. înălțimea subarborelui stâng diferă de a celui drept prin cel puțin o unitate

C. înălțimea subarborelui stâng diferă de a celui drept prin cel mult o unitate ✓correct

D. înălțimea subarborelui drept diferă de a celui stâng prin maxim o unitate ✓correct

Question 304 of 339

Caracteristica unui arbore binar de căutare echilibrat este aceea că:

A. Numărul de noduri al subarborelui stâng diferă de cel al subarborelui drept cu exact o unitate

B. Înălțimea subarborelui stâng diferă de a celui drept cu fix o unitate

C. Înălțimea subarborelui stâng diferă de a celui drept cu cel mult o unitate ✓correct

D. Numărul de noduri al subarborelui stâng diferă de cel al subarborelui drept cu cel mult o unitate

Question 305 of 339

Care dintre următoarele afirmații este adevărată cu privire la arborii AVL?

A. Inserarea unui nod presupune traversarea de la rădăcină către poziția de inserat, urmată de traversarea inversă de la nodul inserat către rădăcină. ✓correct

B. Arborii AVL se traversează o singură dată, de la poziția de inserat către rădăcină.

C. Arborii AVL se traversează o singură dată, de la rădăcină către poziția de inserat.

Question 306 of 339

Care dintre următoarele afirmații este corectă?

- A. Un arbore AVL are gradul de echilibru al fiecărui nod în mulțimea $\{-1, 0, +1\}$
- B. Arborele AVL reprezintă un arbore binar de căutare echilibrat.
- C. Toate variantele sunt corecte ✓ correct**
- D. Rotirea unui arbore se poate face prin rotire la stânga sau la dreapta

Question 307 of 339

Care dintre următoarele afirmații este greșită?

- A. Arborii perfect echilibrați: pentru fiecare nod, diferența dintre numărul de noduri ale subarborelui drept și stâng ia valori în $\{-1, 0, +1\}$
- B. Arborii echilibrați se împart în perfect echilibrați, imperfect echilibrați și inegal echilibrați
- C. Arborii imperfect echilibrați: pentru fiecare nod, diferența dintre înălțimea subarborelui drept și stâng ia valori în $\{-1, 0, +1\}$
- D. Nodurile frunză de pe nivelurile 1..h-2 ale unui arbore perfect echilibrat de înălțime h trebuie să aibă numărul maxim de fii ✓ correct**

Question 308 of 339

Care afirmație legată de caracteristicile arborilor de echilibru este corectă?

- A. Toate variantele sunt corecte ✓ correct**
- B. Factorul de echilibru reprezintă diferența dintre înălțimea subarborelui drept și a celui stâng
- C. Un arbore binar este echilibrat când toți factorii de echilibrare sunt $\{-1, 0, +1\}$
- D. Înălțimea unui arbore de echilibru este lungimea celui mai lung drum de la rădăcină la unul dintre nodurile terminale

Question 309 of 339

Care dintre următoarele proprietăți NU este caracteristică arborilor binari perfect echilibrați?

- A. Orice nod de pe niveluri intermediare are doi fii
- B. Gradul de echilibru al fiecărui nod ia valori în $\{-1, 0, 1\}$
- C. Toate nodurile frunză sunt pe același nivel
- D. Diferența dintre numărul de noduri ale subarborelui drept și cel stâng este 0 ✓ correct**

Question 310 of 339

Dacă gradul de echilibru este -2:

- A. dezechilibru pe partea stângă ✓ correct**
- B. dezechilibru pe partea dreaptă
- C. niciuna dintre variante
- D. este în echilibru

Question 311 of 339

Dezechilibrul în partea dreaptă a unui arbore (calcul de la stânga la dreapta) se rezolvă prin:

- A. rotire la stânga** ✓correct
- B. rotire la dreapta

Question 312 of 339

Echilibrul trebuie verificat după:

- A. doar după inserare
- B. o dată ce este echilibrat, arborele rămâne echilibrat
- C. după fiecare ștergere și inserare** ✓correct
- D. doar după ștergere

Question 313 of 339

Gradul de echilibru pentru un arbore AVL se poate calcula:

- A. doar de la stânga la dreapta
- B. doar de la dreapta la stânga
- C. de la stânga la dreapta și de la dreapta la stânga** ✓correct
- D. de sus în jos

Question 314 of 339

În structura elementelor unui arbore echilibrat găsim:

- A. informația utilă
- B. informația utilă (eticheta), nod stânga, nod dreapta
- C. informația utilă, nod *nod;
- D. informația utilă (eticheta), nod *stânga, nod *dreapta** ✓correct

Question 315 of 339

Înălțimea unui arbore este:

- A. lungimea drumului de la un nod la alt nod
- B. lungimea drumului de la rădăcină la un nod
- C. lungimea celui mai lung drum de la rădăcină la alt nod
- D. lungimea celui mai lung drum de la rădăcină la unul dintre nodurile terminale** ✓correct

Question 316 of 339

Pentru $GE \in \{-2, 2\}$, nodul este:

- A. Echilibrat
- B. Dezechilibrat la dreapta
- C. Dezechilibrat la stânga
- D. Dezechilibrat puternic** ✓correct

Question 317 of 339

Pentru $GE = 0$, nodul este:

- A. Dezechilibrat la dreapta
- B. Dezechilibrat puternic
- C. Echilibrat ✓ *correct*
- D. Dezechilibrat la stânga

Question 318 of 339

Prin câte unități diferă înălțimea subarborelui stâng al unui nod de înălțimea subarborelui drept într-un arbore AVL?

- A. niciuna
- B. cel puțin una
- C. două
- D. cel mult una ✓ *correct*

Question 319 of 339

Proprietatea de echilibru a arborilor AVL este valabilă pentru:

- A. frunze
- B. orice nod ✓ *correct*
- C. cel mai mare element al arborelui
- D. rădăcină

Question 320 of 339

Reechilibrarea unui arbore binar de căutare și păstrarea caracteristicilor AVL se realizează prin:

- A. dublă rotire la dreapta
- B. toate variantele sunt corecte ✓ *correct*
- C. dublă rotire la stânga
- D. rotirea simplă la stânga și rotirea simplă la dreapta

Question 321 of 339

Pentru șirul 23, 8, 28, 10, 5, 3 (rădăcina = 23), pe baza căruia se definește un arbore binar de căutare, care afirmație este greșită?

- A. arborele este dezechilibrat
- B. înălțimea subarborelui drept este egală cu înălțimea subarborelui stâng ✓ *correct*
- C. arborele are $h = 4$
- D. pentru a echilibra arborele trebuie făcută o rotire la dreapta

Question 322 of 339

Se numește factor de echilibrare:

- A. diferența dintre înălțimea subarborelui drept și înălțimea subarborelui stâng ✓correct**
- B. diferența dintre lungimea arborelui stâng și înălțimea acestuia
- C. suma dintre înălțimea subarborelui stâng și lățimea subarborelui drept
- D. suma dintre înălțimea subarborelui drept și înălțimea subarborelui stâng

Question 323 of 339

Structura pentru arborele AVL cuprinde:

- A. informații despre fiul din dreapta și fiul din stânga
- B. informații despre fiul din dreapta, fiul din stânga și despre conținutul nodurilor ✓correct**
- C. informații despre fiul din stânga și despre conținutul nodului
- D. niciun răspuns de mai sus

Question 324 of 339

Arborii heap pot fi de două feluri:

- A. niciuna dintre variante
- B. compleți & incompleți
- C. min-heap & max-heap ✓correct**
- D. sortați & nesortați

Question 325 of 339

Care afirmație este greșită?

- A. O structură heap se poate implementa folosind arbori binari, însă stocarea eficientă se face folosind un masiv bidimensional ✓correct**
- B. Construirea unei structuri heap pornește de la un masiv unidimensional
- C. Proprietatea de structură înseamnă că elementele sunt organizate într-un arbore binar complet
- D. Proprietatea de ordonare (max-heap): valoarea oricărui nod, în afară de rădăcină, este mai mică sau egală decât valoarea părintelui

Question 326 of 339

Care dintre următoarele afirmații este eronată în ceea ce privește o structură heap?

- A. Un min-heap poate fi transformat în max-heap doar inversând relația de ordin
- B. Heap-ul reprezintă un arbore binar ordonat și structurat
- C. Nodurile de pe ultimul nivel al unei structuri heap sunt completate de la dreapta la stânga ✓correct**
- D. Heap-ul poate fi organizat sub formă de min-heap sau max-heap

Question 327 of 339

Ce tip de sortare se poate implementa cu ajutorul structurii binare heap?

- A. bubble sort
- B. heap sort ✓correct**
- C. căutare binară
- D. sortarea rapidă

Question 328 of 339

Conform proprietății de heap (max-heap), elementul maxim din vector este plasat în:

- A. ultimul părinte al heap-ului
- B. ultimul copil al heap-ului
- C. rădăcina heap-ului ✓correct**
- D. primul copil al heap-ului

Question 329 of 339

Cum putem determina care este rădăcina unei structuri heap?

- A. are cea mai mare valoare sau cea mai mică valoare ✓correct**
- B. e setată mereu cu valoarea unu
- C. rădăcina nu are valoare
- D. valoarea sa e de câteva ori mai mare decât celelalte

Question 330 of 339

Dintre proprietățile enumerate, care propoziție este adevărată despre structura heap?

- A. niciuna din variantele de mai sus
- B. valoarea oricărui nod este mai mare sau egală cu valoarea oricărui fiu al său ✓correct**
- C. structura heap se completează de la dreapta la stânga
- D. toate nivelurile sunt completate

Question 331 of 339

Fiul din dreapta se găsește pe poziția (indexare de la 0):

- A. n / p , unde n este numărul total de noduri și p este nodul părinte
- B. $2 * p + 2$, unde p este nodul părinte ✓correct**
- C. $2 * p + 1$, unde p este nodul părinte
- D. $(p - 1) / 2$, unde p este nodul părinte

Question 332 of 339

HeapSort este:

- A. transformă un arbore într-un vector
- B. funcție care returnează valoarea nodului rădăcină
- C. algoritm de sortare a arborelui
- D. algoritm de sortare a unui vector ✓correct**

Question 333 of 339

Într-un MAX-Heap, elementul cu cea mai mare prioritate este mereu pe nodul:

- A. Primul nod din subarborele drept
- B. Nodul rădăcină ✓correct**
- C. Primul nod din subarborele stâng
- D. Nodul frunză

Question 334 of 339

Legat de min-heap, este fals faptul că:

- A. Ultimul nivel al heap-ului conține nodurile cu cele mai mari valori
- B. Cea mai mică valoare se află pe primul nivel al heap-ului
- C. Rădăcina heap-ului va fi elementul cu valoarea cea mai mare ✓correct**
- D. Valorile nodurilor părinte sunt mai mici sau egale decât cele ale copiilor

Question 335 of 339

Pentru parcurgerea unui masiv unidimensional ce stochează nodurile unui heap, NU se poate folosi formula:

- A. Fiu-Dreapta = $2 * (\text{poziție} + 1)$
- B. Fiu-Stânga = $2 * \text{poziție} - 1$ ✓correct**
- C. Fiu-Dreapta = $2 * \text{poziție} + 2$
- D. Fiu-Stânga = $2 * \text{poziție} + 1$

Question 336 of 339

Structura heap poate fi folosită ca:

- A. Niciun răspuns corect
- B. Coadă de prioritate ✓correct**
- C. Stivă
- D. Un vector cu elemente în ordine descrescătoare

Trees

3 questions

Question 337 of 339

In a binary search tree, where are values smaller than a node usually stored?

- A. In the left subtree ✓correct
- B. In the right subtree
- C. Only at the root
- D. Outside the tree

Question 338 of 339

What traversal visits left subtree, node, then right subtree?

- A. In-order traversal ✓correct
- B. Pre-order traversal
- C. Post-order traversal
- D. Level-order traversal

Question 339 of 339

What is the main risk of an unbalanced binary search tree?

- A. Operations can degrade to $O(n)$ ✓correct
- B. It cannot store keys
- C. It always uses no memory
- D. It becomes a hash table

